

COMPUTER ARCHITECTURES

Complete Theory and Exam Guide

Lecture-grounded explanations, plain-English intuition, exam-ready answers, solving recipes, and common traps

COURSE	ASE / Computer Architectures - 2025/26
PREPARED FOR	Marwan - September 2026 exam preparation
SOURCE BASE	All Architecture lectures 00-14, plus supplied past theory prompts

PURPOSE OF THIS MANUAL

This is not a slide dump. It reorganizes the complete lecture set into the exact chain of reasoning usually needed in an exam answer: definition, architecture, operation over time, advantages, limitations, example, and typical variation. The concise answer boxes can be learned for recall; the surrounding sections explain why those answers are correct.

Version: 4 September 2026

How to use this guide

Use the detailed explanation when a topic still feels unclear. Read the plain-English box immediately afterward and verify that you can explain the same mechanism without technical vocabulary. Then close the page and reproduce the exam-ready answer from memory. Finally, use the solving recipe or variation prompts to check whether you can apply the idea rather than merely recognize it.

A good theory answer should normally follow this order: state what the mechanism is and why it exists; describe its hardware or data structures; explain when information is read, written, updated, or released; state the advantages and limitations; finish with a small example or contrast when requested. This order directly matches the structure of many supplied past questions.

MINIMUM-PASS STUDY ORDER

- First: Chapters 5, 10, 11, and 12 - hazards, branch prediction, Tomasulo, CDB, reservation stations, and ROB.
- Second: Chapters 6, 7, 8, and 9 - multicycle pipelines, precise exceptions, ILP, loop unrolling, and cache.
- Third: Chapters 1-4 and 13-15 - foundations, ISA/RISC-V, multiple issue, VLIW, and thread-level parallelism.
- Finish with Chapters 16-18 - model answers, mechanical recipes, and last-day recall.

Exam-priority map from the supplied papers

Priority	Recurring prompts	What you must be able to write
Very high	Hazards and dependencies; BHT/BTB; CDB; reservation stations; ROB; precise exceptions	A complete mechanism answer plus a small example and exact read/write/update timing.
High	Multicycle scheduling; ILP; loop unrolling; branch penalties; dynamic scheduling	Definitions, trade-offs, and a mechanical procedure for tables or code transformations.
Core	Cache organization and mapping; static/dynamic prediction; superscalar/VLIW	Architecture, formulas, advantages/disadvantages, and comparisons.
Supporting	CISC/RISC; performance; power; dependability; ISA; RISC-V; TLP	Reliable definitions and comparison tables; enough context to connect later chapters.

IMPORTANT BOUNDARY

The lecture files are the authority for terminology in this guide. When a broader textbook convention differs, the course convention is stated first. For example, the lectures describe BHT access in decode and BTB access early enough to redirect the PC after fetch; exam answers should follow that wording.

Contents

- 1. Architecture foundations: ISA, CISC, RISC, and design goals
- 2. Quantitative design: performance, Amdahl, power, and dependability
- 3. ISA design and RISC-V essentials
- 4. Pipelining fundamentals
- 5. Dependencies, hazards, forwarding, and control handling
- 6. Multicycle functional units and deep pipelines
- 7. Exceptions, interrupts, and precise state
- 8. Instruction-level parallelism and static techniques
- 9. Cache memories and address calculations
- 10. Branch prediction: BHT, counters, correlating predictors, and BTB
- 11. Dynamic scheduling and Tomasulo
- 12. Hardware speculation, ROB, and in-order commit
- 13. Multiple-issue and superscalar processors
- 14. VLIW and EPIC processors
- 15. Thread-level parallelism and multithreading
- 16. Exam-ready model answer bank
- 17. Mechanical solving recipes
- 18. Last-day recall sheets and self-test
- Appendix A. Full lecture source map

Core abbreviations

Term	Meaning	One-line purpose
CPI	Cycles per instruction	Average cycles used per completed instruction.
ILP	Instruction-level parallelism	Independent instructions that can overlap or execute together.
RAW / WAR / WAW	Read-after-write / write-after-read / write-after-write	Data hazard categories caused by true or name dependencies.
BHT	Branch history table	Predicts taken/not taken from stored history.
BTB	Branch target buffer	Supplies an early predicted target address.
RS	Reservation station	Holds an instruction, operand values, or producer tags until execution.
CDB	Common data bus	Broadcasts a result and tag to all waiting consumers.
ROB	Reorder buffer	Holds speculative results and commits them in program order.
SMT	Simultaneous multithreading	Issues instructions from multiple threads in the same superscalar cycle.

Chapter 1 - Architecture foundations: ISA, CISC, RISC, and design goals

EXAM PRIORITY: SUPPORTING

Build the vocabulary that every later topic assumes.

1.1 What computer architecture includes

The course uses computer architecture in a broad sense. It includes the Instruction Set Architecture, the organization, and the hardware implementation. The ISA is the interface visible to assembly programmers and compilers: instructions, registers, data types, addressing modes, and the programmer-visible behavior. Organization describes how the ISA is implemented, for example with pipelines, caches, functional units, buses, and control logic. Hardware is the concrete technology and circuits used to realize that organization.

A computer architect is therefore not merely choosing instructions. The architect must satisfy functional requirements while balancing price, power, performance, and dependability. Different markets assign different weights to these objectives. Mobile devices emphasize energy efficiency; desktops emphasize price/performance; servers and warehouse-scale computers emphasize throughput, scalability, and availability; embedded systems may impose real-time, memory, power, and reliability constraints.

PLAIN ENGLISH

ISA says what the machine promises to do. Organization says how the inside is arranged to do it. Hardware says what physical implementation is used. Two processors may implement the same ISA with very different pipelines and caches.

EXAM-READY ANSWER

Computer architecture covers three levels: the ISA, which is the programmer/compiler-visible interface; the organization, which defines the internal datapath, control, memories, and execution structures; and the hardware technology implementing them. The design must meet functionality, cost, power, performance, and dependability constraints. The relative importance of these constraints depends on the product market.

1.2 Register-memory and register-register machines

Processors can be classified according to where operands are held and which instructions may access memory. In a register-memory architecture, an ALU instruction can combine registers, immediate values, and possibly memory operands. In a register-register architecture, also called a load-store architecture, memory is accessed only through explicit load and store instructions; ALU instructions operate on registers and immediate values.

Load-store organization simplifies the datapath and instruction timing because an ALU operation does not also have to perform an arbitrary memory access. The price is that a source-level operation may require more assembly instructions: load operands, execute the operation, then store the result. Register-memory instructions can produce denser code, but instruction order may strongly affect the number of memory accesses and therefore performance.

Feature	Register-memory	Register-register / load-store
Memory operands	Some ALU instructions may directly read or write memory.	Only load/store instructions access memory.
ALU operands	Registers, immediates, and possibly memory.	Registers and immediates.
Instruction count	Often lower for a source-level task.	Often higher because explicit transfers are needed.
Implementation	More complex decoding and execution cases.	More regular datapath and easier pipelining.
Example families	x86 is the main course example.	RISC-V, ARM, and MIPS are course examples.

Same task, different architectural style

```
# Register-memory style idea
MOV AL, [SI]
ADD AL, [BX]

# Load-store RISC-V equivalent idea
lb  t1, 0(t0)
lb  t3, 0(t2)
add t4, t1, t3
```

1.3 CISC and RISC

CISC means Complex Instruction Set Computer. Its historical design goal is to express a task in relatively few assembly instructions. CISC machines commonly support many instruction forms, rich addressing modes, variable instruction length, and instructions whose execution time can vary. This tends to reduce code size but increases the complexity of fetch, decode, and execution.

RISC means Reduced Instruction Set Computer. Its design goal is a simpler, more regular processor. Instructions are typically simpler and commonly fixed length; execution is organized around registers and explicit loads/stores. The simplified structure supports efficient pipelining and normally lowers the number of cycles required by a simple instruction, though more instructions may be needed and code can be longer.

The distinction is a design tendency, not a rule that every modern processor follows perfectly. For the exam, use the characteristics stated in the lectures: CISC emphasizes compact code and complex/variable instructions, while RISC emphasizes simple/fixed instructions and simpler implementation.

Dimension	CISC	RISC
Main objective	Complete a task with fewer assembly lines.	Keep the processor and instruction set regular and simple.
Instruction length	Typically variable.	Typically fixed in the base ISA.
Instruction complexity	Rich operations and addressing combinations.	Simple operations, often load-store.
Cycles per instruction	Can vary widely.	Normally lower for simple instructions; optimistic goal near one.
Code size	Usually smaller.	Usually larger.
Hardware	More complex fetch/decode/control.	Simpler and more pipeline-friendly.

COMMON TRAP

Do not write that RISC automatically means a faster whole program. Program time depends on instruction count, CPI, and clock period. A RISC program may execute more instructions, while a CISC implementation may internally translate complex instructions into simpler operations.

EXAM-READY ANSWER

CISC architectures aim to implement a source-level task with few and powerful instructions. They commonly use variable-length instructions, many addressing modes, variable execution times, complex control, and compact code. RISC architectures use a smaller and more regular set of simple instructions, normally fixed-length encoding and a load-store organization. This simplifies decoding and pipelining and often reduces CPI, but can increase the instruction count and code size.

1.4 The governing design principle: make the common case fast

Instruction frequencies are highly non-uniform. Loads, stores, simple arithmetic, logical operations, comparisons, branches, calls, and returns account for a large part of execution. The architect should optimize these frequent cases instead of spending excessive cost on rare operations. The same principle reappears throughout the course: caches exploit frequent locality, forwarding accelerates common RAW dependences, branch prediction targets frequent control transfers, and dynamic scheduling lets independent work bypass uncommon long delays.

Lecture grounding: 00 - CISC-RISC difference.pdf, slides 2-27; 01 - computer architecture.pdf, slides 20-40 and 31-33; 02 - introduction to isa.pdf, slides 2-9 and 34-35.

Chapter 2 - Quantitative design: performance, Amdahl, power, and dependability

EXAM PRIORITY: CORE

Know the equations, but also state exactly what each metric measures.

2.1 Response time and throughput

Performance has two main viewpoints. A user normally cares about response time, the time between starting and completing one operation. A system manager often cares about throughput, the total amount of work completed per unit time. An enhancement may improve one without improving the other by the same amount. For example, adding processors can increase total throughput even when the latency of one sequential task barely changes.

Elapsed time is wall-clock time and includes CPU execution, operating-system activity, waiting for I/O, and time spent on other processes. CPU time counts time actually spent by the processor on the program. User CPU time is time in the program itself; system CPU time is time in operating-system services on its behalf. State which time is being compared before claiming that one computer is faster.

PLAIN ENGLISH

Response time asks: how long did my job take? Throughput asks: how many jobs can the system finish in a second? They are related, but not identical.

2.2 CPU performance equation

CORE FORMULA

CPU time = sum over instruction classes of $(IC_i \times CPI_i) \times \text{clock-cycle time}$. Equivalently, CPU time = total instruction count $\times$ average CPI $\times$ clock-cycle time, where average CPI is weighted by the executed instruction mix.

Instruction count is influenced mainly by the ISA, compiler, algorithm, and input. CPI is influenced by the organization and by dynamic events such as dependences, branch outcomes, and cache hits or misses. Clock-cycle time is influenced by hardware technology and by the longest timing path in the organization. These quantities interact: a design that shortens the clock period may need a deeper pipeline and therefore incur a larger branch penalty; an instruction that reduces instruction count may be more complex and increase CPI or clock time.

In a pipelined processor, it is dangerous to assign one immutable CPI to each opcode. The same load can be a cache hit or miss; the same arithmetic instruction can execute without a stall or wait behind a dependence. Analytical evaluation therefore needs the dynamic instruction sequence and memory behavior.

Change	IC effect	CPI effect	Clock-period effect
Richer instruction	May reduce IC.	May increase due to complexity.	May lengthen critical path or require more stages.
Deeper pipeline	Usually unchanged.	Ideal CPI unchanged, but hazards may raise it.	Can reduce period if stages are balanced.
Better cache	Usually unchanged.	Reduces memory-stall contribution.	May slightly affect hit time/critical path.
Better compiler schedule	May slightly change IC.	Can remove stalls.	Normally unchanged.

2.3 Speedup and Amdahl's law

Speedup is the old execution time divided by the new execution time. Amdahl's law states that the benefit of an enhancement is limited by the fraction of execution time that can use it. Let f be the originally measured fraction improved and S the speedup of that fraction. The new normalized execution time is $(1 - f) + f/S$, so the overall speedup is $1 / ((1 - f) + f/S)$.

The fraction must refer to original execution time, not instruction count unless the problem explicitly makes them equivalent. Even an infinitely fast enhancement leaves the unenhanced fraction, so the maximum speedup is $1/(1-f)$.

WORKED EXAMPLE

An enhancement makes 40% of execution ten times faster. New normalized time = $0.60 + 0.40/10 = 0.64$. Overall speedup = $1/0.64 = 1.5625$. The answer is not 4x and not 10x because 60% of the original time is untouched.

EXAM-READY ANSWER

Amdahl's law evaluates an enhancement that accelerates only part of a computation. If fraction f of the original execution time is accelerated by S , the overall speedup is $1/((1-f)+f/S)$. It shows that the unenhanced fraction limits total performance: even as S tends to infinity, speedup cannot exceed $1/(1-f)$. Therefore, optimizing a common operation usually gives more benefit than a very large acceleration of a rare operation.

2.4 Benchmarks and fair comparison

A benchmark workload should represent the programs and inputs that matter to the intended users. The lecture distinguishes real programs, kernels, toy benchmarks, synthetic benchmarks, and benchmark suites. Suites reduce the chance that one narrow component dominates the result, but the programs and inputs must still be reported so measurements are reproducible.

Execution times can be summarized using total time, arithmetic mean, or a weighted arithmetic mean. Weights should represent the frequency or importance of programs in the real workload. A single unqualified average can hide very different behavior across programs, so always state the chosen metric and reference machine when normalized values are used.

2.5 Power and energy

COURSE EQUATIONS

Dynamic power per switching element is modeled as $P_{\text{dynamic}} = 1/2 \times C \times V^2 \times f$. Static power is $P_{\text{static}} = V \times I$. Dynamic energy per switching event/work unit is proportional to $C \times V^2$.

Dynamic power grows linearly with switching frequency and approximately with the square of supply voltage. Lowering voltage is therefore powerful, although it can reduce achievable frequency. Static power comes from leakage current and becomes significant as devices scale. Power matters for cooling and peak delivery; energy is power integrated over time and is especially important for battery-powered devices. A faster design can consume more power yet sometimes less total energy if it finishes sufficiently sooner, so do not use the terms interchangeably.

2.6 Dependability

Dependability is the ability of a system to deliver correct service. It can be reduced by hardware design bugs, software bugs, manufacturing defects, and faults during operation. The course uses Mean Time To Failure (MTTF), Failures In Time (FIT), Mean Time To Repair (MTTR), Mean Time Between Failures (MTBF), and availability. One FIT means one failure in one billion hours. The stated relation is $MTBF = MTTF + MTTR$. Availability is the probability that the system is operating correctly at a generic instant; for a repairable system it is commonly related to uptime divided by uptime plus repair time.

Metric	Meaning	Interpretation
MTTF	Mean time from correct operation to failure.	Higher is better.
FIT	Failure rate unit; reciprocal-style reliability measure.	Lower is better; 1 FIT = one failure per 10^9 device-hours.
MTTR	Mean time required to repair and restore service.	Lower is better.
MTBF	Mean time between successive failures.	Course relation: $MTTF + MTTR$.
Availability	Probability the service is working at a random time.	Improved by long MTTF and short MTTR.

EXAM-READY ANSWER

Dependability is the ability to deliver correct service. It is affected by design bugs, software errors, manufacturing defects, and operational faults. MTTF measures average operating time before failure; FIT expresses failure rate; MTTR measures average repair time; MTBF is $MTTF + MTTR$ in the course model. Availability is the probability that the system is operational at a generic instant and improves when failures are rare and repair is fast.

Lecture grounding: 01 - computer architecture.pdf, slides 38-74. Numerical definitions and equations follow the notation used in the lecture.

Chapter 3 - ISA design and RISC-V essentials

EXAM PRIORITY: CORE

Understand the trade-offs that shape instruction formats and compiler-visible behavior.

3.1 ISA as the programmer/compiler interface

The ISA specifies the operations, programmer-visible registers, supported data types, memory addressing rules, addressing modes, instruction formats, and control-transfer behavior. ISA choices affect processor performance and complexity, compiler complexity, code size, and power. No single choice is best for every market.

Modern machines are General-Purpose Register machines because registers are faster than memory and easier for a compiler to manage. A load-store ISA further regularizes execution by separating memory transfers from computation.

3.2 Endianness and alignment

Endianness defines how the bytes of a multi-byte value are arranged in increasing memory addresses. In little-endian organization, the lowest-address byte contains the least-significant byte. In big-endian organization, the lowest-address byte contains the most-significant byte. Endianness does not reverse the bits inside a byte and does not change the numerical value held in a register; it changes byte order in memory.

An access is naturally aligned when its address is a multiple of the data size, for example a 4-byte word at an address divisible by four. Supporting misaligned accesses gives programmers flexibility but may require multiple memory transfers, alignment hardware, and extra cycles. Restricting accesses to aligned addresses simplifies the implementation.

COMMON TRAP

For little endian, the least-significant byte goes to the lowest address. Do not write that the whole binary string is reversed. Alignment is about the address being a multiple of the object size, not about the value stored.

3.3 Addressing modes

Mode	Effective operand/address idea	Typical use
Register	Operand is in a named register.	Fast ALU operands.
Immediate	Constant is encoded in the instruction.	Small constants and masks.
Displacement	EA = register + signed constant.	Stack fields, arrays, structures.
Register indirect	EA = register; displacement is zero.	Pointers.
Indexed / scaled	EA combines base and index, possibly scaled.	Array element addressing.
Direct / absolute	Address is encoded or formed relative to zero.	Known static locations.
Memory indirect	Instruction points to a memory cell containing the final address.	Pointer chains; complex ISA.
Post-increment	Use current address, then update base.	Sequential traversal.
Pre-decrement	Update base, then use new address.	Stacks and reverse traversal.

Adding addressing modes can reduce instruction count and code size, but it increases decode/datapath complexity and may increase average CPI. The lecture data show that displacement, immediate, and register-indirect modes cover most accesses, supporting the principle of keeping the common modes efficient. A displacement field of roughly 12-16 bits and an immediate field of at least 8-16 bits cover a large fraction of typical uses according to the cited measurements.

3.4 Control flow and procedures

Control transfers include conditional branches, unconditional jumps, procedure calls, and returns. Conditional branches are the most frequent. A PC-relative target uses a signed displacement from the current PC, reducing encoding bits because targets are commonly near and making code position independent. Register-indirect jumps are needed when a target is not known at compile time, for example switch tables, dynamic libraries, and virtual-function dispatch.

A procedure call must preserve a return address and follow a convention for registers. Caller-saved registers may be overwritten by the callee, so the caller saves any live values it needs. Callee-saved registers must be restored by a function that changes them. These rules make separately compiled modules interoperable.

3.5 Instruction encoding trade-offs

Encoding style	Benefits	Costs
Highly variable length / flexible operand count	Minimum code size and rich combinations.	Complex fetch/decode; lower regularity and often lower performance.
Fixed length / fixed fields	Simple parallel decode, alignment, and high performance.	Larger code and less encoding flexibility.
Multiple regular formats	Compromise between code size and simple decoding.	More formats to implement, but still structured.

The compiler and ISA designer must cooperate. Register allocation is a central compiler optimization and is easier with a sufficiently large register set. The lecture recommends at least 16 registers, orthogonality, and simplicity. Regular primitives are preferable to highly specialized solutions because the compiler can combine primitives in different ways.

3.6 RISC-V design profile

RISC-V is an open-standard, modular ISA. A base integer ISA is combined with optional extensions. The course presents 32-, 64-, and 128-bit address/data variants, with a fixed 32-bit base instruction encoding and optional 16-bit compressed encodings. The default memory system is little endian. RISC-V is load-store and mainly uses immediate, displacement, and register-indirect addressing.

The integer register file has 32 registers. x0 is hardwired to zero, enabling moves, constant creation, and discarded results through ordinary instructions. The ABI gives conventional roles such as ra for return address, sp for stack pointer, a0-a7 for arguments, t registers for temporaries, and s registers for saved values.

Format	Main purpose	Essential fields
R-type	Register-register ALU operations.	opcode, rd, rs1, rs2, funct3, funct7.
I-type	Immediate ALU operations, loads, and indirect jumps.	opcode, rd, rs1, funct3, 12-bit immediate.
S-type	Stores; immediate split across the instruction.	opcode, rs1, rs2, funct3, two immediate portions.
B-type	Conditional branches; split PC-relative immediate.	opcode, rs1, rs2, funct3, branch immediate portions.
U-type	Upper immediate operations.	opcode, rd, 20-bit upper immediate.
J-type	Long PC-relative jump and link.	opcode, rd, jump immediate portions.

COURSE-LEVEL RISC-V FACTS TO REMEMBER

- Main memory is accessed only by loads and stores.
- Loads can sign-extend or zero-extend smaller data; stores write only the selected low-order bytes.
- Immediate and displacement fields are commonly 12-bit signed values in the described formats.
- jal writes a return address while changing the PC; jalr uses a register-based target.
- Conditional branches compare two registers and include signed and unsigned variants.
- Pseudo-instructions such as li, j, call, ret, and nop are expanded by the assembler into real instructions.

Small instruction-role summary

```

addi x1, x0, 10      # x1 = 10
lw   x2, 32(x3)      # x2 = MEM[x3 + 32]
add  x4, x1, x2       # register-register ALU
sw   x4, 0(x3)        # MEM[x3] = x4
beq  x4, x0, label    # branch if x4 == 0
jal  x1, function     # call: save return address in x1

```

EXAM-READY ANSWER

RISC-V is an open, modular load-store ISA. The base instructions are regular and normally 32 bits long; optional extensions add features such as multiplication, floating point, vectors, or compressed encoding. It uses 32 general-purpose registers, with x0 fixed to zero, and primarily immediate, displacement, and register-indirect addressing. Regular instruction formats place opcode, register fields, function fields, and immediates in predictable positions, simplifying decode and compiler generation.

Lecture grounding: 02 - introduction to isa.pdf, slides 2-60; 03 - RISC-V.pdf, slides 2-46; 00 - CISC-RISC difference.pdf, slides 25-27.

Chapter 4 - Pipelining fundamentals

EXAM PRIORITY: HIGH

Explain overlap, stage timing, speedup limits, and the canonical five-stage datapath.

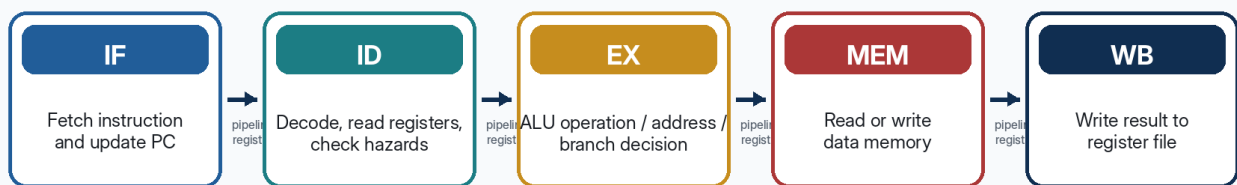
4.1 What pipelining changes

Pipelining divides instruction processing into stages separated by pipeline registers. Different instructions occupy different stages in the same clock cycle, so their executions overlap. The machine cycle is the common clock period, determined by the slowest stage plus pipeline-register and clocking overhead.

Pipelining primarily increases throughput, not the latency of one instruction. A single instruction still traverses all stages and may take slightly longer than in an unpipelined design because of register overhead and imperfect stage balance. In the ideal steady state, a k -stage pipeline completes one instruction per cycle, so CPI approaches one. The ideal speedup approaches k only for a long instruction stream with balanced stages and no hazards.

Canonical 5-stage pipeline

Instructions overlap; pipeline registers separate the stages.



Ideal steady state: one instruction completes per cycle (CPI approx. 1), but each instruction still crosses all stages. Clock period is limited by the slowest stage plus pipeline-register overhead.

Figure 4.1 - Canonical five-stage pipeline and the role of each stage.

PLAIN ENGLISH

A laundry pipeline does not wash one shirt instantly. It lets washing, drying, and folding happen on different shirts at the same time. Throughput improves even though each shirt still passes through every step.

4.2 Ideal timing equations

PIPELINE TIMING

Let k be the number of stages and n the number of instructions. Ignoring hazards, total cycles = $k + n - 1$. T_{clock} is approximately $\max(\text{stage delay}) + \text{pipeline-register overhead}$. Total time = $(k + n - 1) \times T_{\text{clock}}$. For large n , throughput tends to one result per T_{clock} .

For an unpipelined implementation, the clock or operation time must cover the sum of the required stage delays for an instruction. Pipelining permits a shorter clock based on the maximum individual stage delay. Speedup is reduced when stages are unbalanced, the instruction stream is short, pipeline-register overhead is large, or hazards introduce bubbles.

4.3 The five stages

Stage	Main actions	Values produced for later stages
IF - Instruction Fetch	Read instruction memory; form next sequential PC.	Instruction and PC-related information.
ID - Decode/Register Fetch	Decode; read register operands; form/sign-extend immediate; detect hazards.	Operand values, register identifiers, immediate, control signals.

Stage	Main actions	Values produced for later stages
EX - Execute/Address	ALU operation; effective address; branch target and/or condition.	ALU result, address, branch decision.
MEM - Memory	Read or write data memory; pass non-memory ALU result.	Loaded data or forwarded ALU result.
WB - Write Back	Write the selected result to the register file.	Architectural register update.

Pipeline registers must carry both data and control information between stages. The basic design needs enough memory bandwidth to fetch an instruction while another instruction accesses data. This is often achieved through separate instruction and data caches. The register file is commonly organized so a write in WB occurs early in the cycle and reads in ID occur later, allowing an instruction in ID to observe a value written in the same cycle.

4.4 Throughput, latency, and stage depth

Concept	Correct statement	Frequent wrong statement
Throughput	Completions per unit time; ideally one instruction per cycle after filling.	Every instruction takes one cycle.
Instruction latency	Time from its fetch to completion across all stages.	Equal to the clock period.
Pipeline fill/drain	First result appears only after the stages fill; final cycles drain remaining instructions.	n instructions always take n cycles.
Pipeline depth	More stages may permit a shorter clock, but raise overhead and branch/load penalties.	More stages always improve performance.
CPI	Ideal steady-state CPI is approximately one for single issue, but stalls increase it.	Pipelining guarantees CPI exactly one.

EXAM-READY ANSWER

Pipelining divides instruction execution into stages separated by registers and overlaps different instructions. The clock period is set by the slowest stage plus register overhead. It increases throughput: after the pipeline is filled, an ideal single-issue design completes one instruction per cycle. It does not reduce the latency of one instruction, and real speedup is below the stage count because of fill/drain time, unbalanced stages, register overhead, and structural, data, and control hazards.

4.5 What creates a hazard

A hazard is a situation in a particular pipeline organization that prevents the next instruction from executing in its intended cycle while preserving correctness. Structural hazards come from resource conflicts; data hazards come from overlapping reads and writes constrained by dependencies; control hazards come from an unresolved next PC. The following chapter treats each in detail.

Lecture grounding: 04 - pipeline.pdf, slides 2-25 and the five-stage datapath sequence in slides 7-20. The ideal and real-performance distinctions follow slides 4-5 and 21-25.

Chapter 5 - Dependencies, hazards, forwarding, and control handling

EXAM PRIORITY: VERY HIGH

This is one of the most repeated theory areas: distinguish program dependencies from machine hazards.

5.1 Dependency is not the same as hazard

A dependency is a property of the program and defines a required relation among instructions. It constrains the legal order in which values are produced, names are reused, or instructions are controlled by branches. A hazard is a problem created when a particular pipeline overlaps instructions in a way that could violate one of those requirements. A dependency can exist without causing a stall: forwarding may satisfy a true data dependence in time. Conversely, the same program on a different pipeline may experience a hazard because values are read or written at different stages.

Dependencies set an upper bound on exploitable parallelism. Hazards determine what the actual hardware must detect, bypass, delay, or recover from.

Dependencies are program properties; hazards depend on the machine

A dependency creates the possibility of a hazard. A hazard may or may not require a stall.

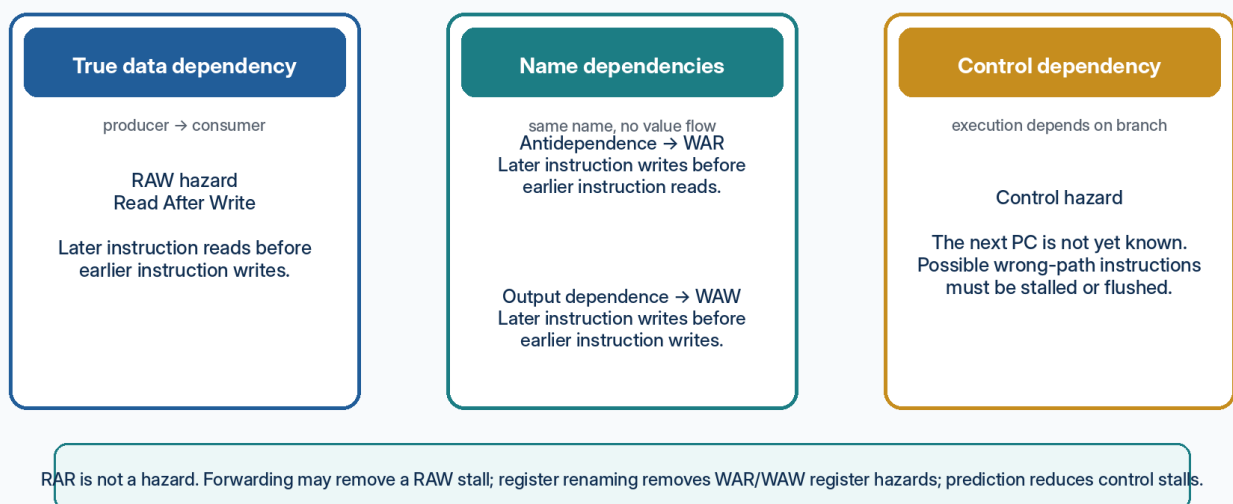


Figure 5.1 - Program dependencies and the hazard categories they may create.

EXAM-READY DISTINCTION

Dependencies belong to the code; hazards belong to the interaction between the code and a specific pipeline. A dependency creates the possibility of a hazard, but the hazard may be avoided by organization features such as forwarding or register renaming, and may therefore produce no stall.

5.2 Three dependency classes

True data dependency

Instruction j is truly data dependent on instruction i when i produces a value that j consumes. The relation is transitive through chains of dependent instructions. This is real information flow and cannot be removed by renaming. The consumer must receive the value produced by the correct earlier instruction.

```
add x1, x2, x3      # produces x1
sub x4, x1, x5      # consumes x1: true data dependency
```

Name dependencies

A name dependency exists when instructions use the same register or memory location name without a value flowing from the earlier instruction to the later one. An antidependence occurs when an earlier instruction reads a name and a later instruction writes it. An output dependence occurs when both instructions write the same name. These relations can be removed by assigning a different register name, statically by the compiler or dynamically by hardware.

```
# Antidependence: earlier read, later write
add x4, x1, x2      # reads x1
sub x1, x5, x6      # writes x1

# Output dependence: both write x1
add x1, x2, x3
sub x1, x4, x5
```

Control dependency

An instruction is control dependent on a branch when whether it should execute depends on that branch. Preserving every control dependency is sufficient for correctness, but not always necessary. An instruction may be moved speculatively across a branch if doing so preserves exception behavior and data flow. A safe instruction whose result is dead on the path where it should not have executed can sometimes be moved.

```
beq x1, x0, L
add x4, x5, x6      # execution depends on the branch outcome
L: ...
```

5.3 Hazard classes

Structural hazards

A structural hazard occurs when two simultaneous operations require the same hardware resource and that resource cannot serve both. Examples include a single memory used for both instruction fetch and a data access, one unpipelined divider requested by two instructions, or a single register-file write port needed by two instructions completing in the same cycle. Solutions are to duplicate or pipeline the resource, add ports, or stall one instruction. The choice is a cost/performance trade-off.

RAW - Read After Write

A RAW hazard occurs when a later instruction attempts to read a source before an earlier instruction has produced or written the required value. It corresponds to a true data dependency. RAW is the common data hazard in an ordinary in-order five-stage pipeline.

```
add x1, x2, x3
sub x4, x1, x5      # must not use the old x1
```

WAW - Write After Write

A WAW hazard occurs when a later instruction writes a destination before an earlier instruction writes the same destination, reversing the required final-write order. It corresponds to an output dependence. WAW cannot occur in the basic five-stage in-order pipeline when every instruction writes only in WB and reaches WB in order. It can occur with variable-latency units, multiple write stages, or out-of-order completion.

```
fld    f1, 0(x2)      # long path to write f1
fadd.d f1, f2, f3     # could complete and write f1 first
```

WAR - Write After Read

A WAR hazard occurs when a later instruction writes a name before an earlier instruction has read that name. It corresponds to an antidependence. It requires a machine that can write early or allow the later instruction to overtake an earlier operand read, so it is absent from the conventional in-order five-stage pipeline but can appear with out-of-order execution or complex addressing modes.

```
fadd.d f6, f0, f8     # must read old f8
fsub.d f8, f9, f14    # later write must not happen first
```

RAR - Read After Read

RAR is not a hazard because two reads do not change architectural state and their order does not affect the values, assuming no other write intervenes.

Hazard	Underlying relation	When possible	Main remedy
RAW	True data dependency	Consumer reads before producer makes value available.	Forwarding when timing permits; otherwise stall.
WAR	Antidependence	Later write overtakes earlier read.	Register renaming or enforce order.
WAW	Output dependence	Later write overtakes earlier write.	Register renaming, write-order control, or stall.
Structural	Resource conflict	Two operations need an unavailable shared resource.	Duplicate/pipeline resource or stall.
Control	Control dependency	Next PC or execution path is unresolved.	Stall, predict, flush, or delayed branch.

5.4 Stalls and bubbles

A stall delays one or more instructions until the hazard disappears. In a simple interlocked pipeline, the PC and IF/ID register can be frozen while the control signals entering the next stage are changed to represent a no-operation. That no-operation becomes a bubble that moves down the pipeline. Older instructions are allowed to continue so they can produce the value or release the resource causing the stall.

A bubble is not a real program instruction; it is an empty pipeline slot. The number of stall cycles is determined by when the consumer needs the value and when the producer can make it available, including available forwarding paths.

5.5 Forwarding

Forwarding, or bypassing, sends a result directly from a later pipeline register or functional-unit output to the input that needs it, rather than waiting for the value to be written to and reread from the register file. Control logic compares destination registers of older instructions with source registers of the instruction in the consuming stage. Multiplexers select the register-file value or a forwarded value from locations such as EX/MEM or MEM/WB.

Forwarding preserves the true dependency; it only provides the correct value earlier. Paths may be needed toward ALU inputs, store-data inputs, and branch-comparison logic. Register x0 or an instruction that does not write a register must not trigger forwarding.

EXAM-READY ANSWER: FORWARDING

Forwarding reduces RAW stalls by routing a result directly from the stage that produced it to the stage that consumes it, without waiting for WB and a later register-file read. Hazard logic compares pending destination registers with the current source registers and selects an EX/MEM or MEM/WB bypass through multiplexers. Forwarding cannot remove a hazard when the value becomes available after the cycle in which the consumer needs it.

5.6 Why the load-use hazard still stalls

For an ALU instruction, the result is normally available at the end of EX and can be forwarded to the next instruction's EX inputs in the following cycle. For a load, the requested data appears only at the end of MEM. An immediately following consumer would need the operand at the beginning of its EX stage in that same cycle. Forwarding cannot send a value backward in time, so one bubble is required in the standard five-stage organization.

```
lw  x1, 0(x2)
add x3, x1, x4      # immediate load-use: normally one stall

# Compiler scheduling can fill the gap with an independent instruction:
lw  x1, 0(x2)
add x7, x8, x9      # independent
add x3, x1, x4
```

MECHANICAL TIMING QUESTION

Ask two questions: In which stage does the producer make the value available? In which stage does the consumer need it? A stall is needed whenever the consumer would need the value before a valid forwarding path can deliver it.

5.7 Control hazards and branch solutions

A control hazard occurs because the processor fetches sequential instructions before knowing whether a branch is taken and what the target PC is. If the branch decision is made in EX, the instructions currently in IF and ID may belong to the wrong path. When the branch is taken, they must be invalidated, giving a two-cycle penalty in the basic example. Moving condition evaluation and target generation earlier reduces the penalty but adds hardware and may create new forwarding needs.

The lecture presents four main approaches. Freezing waits until the decision is known but always loses cycles. Predict-not-taken continues sequentially and flushes on a taken branch. Predict-taken starts at a known target and flushes when the branch is not taken. A delayed branch defines one or more architecturally executed delay-slot instructions after the branch; the compiler tries to place useful work there that is safe for both outcomes.

Technique	Behavior	Cost/limitation
Freeze	Stop fetching/issuing until branch outcome is known.	Simple but pays the full delay for every branch.
Predict not taken	Fetch fall-through; keep it if NT, flush if T.	Good only when NT is common; taken penalty remains.
Predict taken	Fetch target as soon as target is available.	Needs early target; NT branches flush.
Delayed branch	Instruction(s) after branch execute regardless of outcome.	Compiler must fill safely; poor fit for deep pipelines.
Dynamic prediction	Use run-time history and target structures.	Higher accuracy but more hardware and recovery logic.

BRANCH-DELAY-SLOT WORDING USED IN EXERCISES

When an exercise says the branch delay is one cycle but the delay slot is not enabled, the sequential instruction may still appear in the pipeline because it was fetched, but it is not architecturally guaranteed to execute. If the branch redirects control, that wrong-path instruction is flushed. An enabled delay slot means the instruction immediately after the branch is executed regardless of the branch outcome.

5.8 Full exam-ready answer: hazards

MODEL ANSWER

A pipelined processor can experience structural, data, and control hazards. A structural hazard occurs when concurrent instructions need the same unavailable resource, for example one memory or one write port; it is solved by adding resources or stalling. Data hazards are RAW, WAR, and WAW. RAW corresponds to a true data dependency and occurs when a later instruction reads before an earlier one writes; forwarding can often remove the stall, except when the value is available too late, as in an immediate load-use pair. WAR corresponds to an antidependence and occurs when a later write overtakes an earlier read. WAW corresponds to an output dependence and occurs when a later write overtakes an earlier write. WAR and WAW do not occur in the basic in-order five-stage pipeline but may occur with variable latency or out-of-order execution; register renaming removes register name hazards. A control hazard occurs when a branch leaves the next PC unknown; freezing, prediction with flushing, early branch resolution, or delayed branches can reduce its effect.

Lecture grounding: 04 - pipeline.pdf, slides 25-87; 07 - ilp.pdf, slides 21-48. Past-paper emphasis: hazards/dependencies questions in the supplied February 2026 sessions.

Chapter 6 - Multicycle functional units and deep pipelines

EXAM PRIORITY: HIGH

Variable EX latency creates longer waits, new structural conflicts, and out-of-order completion.

6.1 Why floating-point operations need multiple cycles

Floating-point addition, multiplication, and especially division require more work than a simple integer ALU operation. Forcing every operation into one cycle would require a slow clock or very complex hardware. A common design therefore makes EX contain several functional units with different numbers of stages or different occupancy times.

A unit can be pipelined, allowing a new operation to start before the previous one finishes, or unpipelined, keeping the unit busy for the whole operation. These two properties are captured by latency and initiation interval.

6.2 Latency and initiation interval

Quantity	Course definition	Consequence
Latency	Number of cycles that must separate a producing instruction from a dependent consumer.	Determines RAW spacing and forwarding timing.
Initiation interval	Minimum cycles between starting two operations of the same type on the same unit.	Determines structural conflicts between independent operations.

A pipelined FP adder may have latency three yet initiation interval one: a dependent consumer waits, but independent additions can start in consecutive cycles. An unpipelined divider may have both a long latency and an initiation interval approximately equal to its full running time, so a second divide cannot start until the first releases the unit.

Example unit from lecture	Latency	Initiation interval	Interpretation
Integer ALU	0	1	Back-to-back dependent use can be satisfied by normal forwarding.
Data memory	1	1	Immediate load-use requires one-cycle separation.
FP add	3	1	Long dependent wait, but a new independent add each cycle.
FP/integer multiply	6	1	Deeply pipelined multiplier.
FP/integer divide	24	25	Long and effectively unpipelined in the example.

6.3 New structural hazards

An unpipelined divider can be requested by a second divide while still occupied. Variable execution times can also cause several instructions to reach MEM or WB in the same cycle, requiring more than one register write. Adding write ports is expensive because the register file, bypass network, and arbitration grow. Alternatives are to stall an instruction in ID before it enters the conflicting path or stall it before MEM/WB.

6.4 Longer RAW stalls and new WAW hazards

Longer operation latency increases the distance needed before a dependent consumer. Because instructions no longer necessarily reach WB in order, a later short instruction may write the same destination before an earlier long instruction, creating WAW. A simple in-order issue design can prevent this by checking the destination of the instruction in ID against destination registers of all still-pending instructions and stalling on a match.

WAR does not normally arise if every instruction reads operands in ID before later instructions are issued. It becomes relevant when operand reading itself can occur out of order, as in dynamic scheduling.

ID-STAGE CHECKS IN THE LECTURE

- Structural: is the required functional unit or write-back resource available at the required time?
- RAW: is any source register produced by a pending instruction, and will its result be available when needed?
- WAW: does the current instruction have the same destination as any still-active instruction in a multicycle stage?

6.5 Deep pipelines and the MIPS R4000 example

The lecture uses the MIPS R4000 to illustrate a deeper eight-stage pipeline, sometimes called a superpipeline. Instruction fetch and data access are each split across multiple stages to permit a higher clock frequency despite slower cache access. A new instruction can still begin every cycle because those memory paths are themselves pipelined.

Deeper pipelines require more forwarding paths and increase the number of cycles between a load and its consumer or between a branch and the correct next instruction. In the course example, the load delay slot grows to two cycles and the branch delay slot to three cycles. This shows the central trade-off: a shorter clock can be offset by a larger penalty measured in cycles.

EXAM-READY ANSWER

Multicycle functional units are used because complex operations cannot efficiently finish in one short clock period. Latency is the separation required before a dependent consumer; initiation interval is the separation between operations using the same unit. A pipelined unit can have long latency but initiation interval one, whereas an unpipelined divider remains busy for many cycles. Variable latency increases RAW stalls, can create structural conflicts at the unit or write port, and can allow out-of-order completion, hence WAW hazards. Hazard detection must check resource availability, source readiness, and pending equal destinations.

6.6 Pipeline-table solving checklist

1. Write the exact stage sequence for each instruction type, including every FP substage.
2. Mark when each source operand is needed and when each producer value becomes forwardable.
3. Respect the functional unit initiation interval, not just its latency.
4. Check whether two instructions would require the single WB port in the same cycle.
5. Check WAW against all older unfinished writers before allowing issue.
6. If EX completion may be out of order, do not force an independent short operation to wait behind a long one unless issue/resource rules require it.

Lecture grounding: 05 - multicycle.pdf, slides 2-42; 04 - pipeline.pdf for forwarding and control-hazard foundations.

Chapter 7 - Exceptions, interrupts, and precise state

EXAM PRIORITY: VERY HIGH

The key exam idea is not merely listing causes; it is showing how the machine preserves restartable program order.

7.1 Definition and terminology

An exceptional event changes the normal execution order. The lecture commonly uses exception for an internal CPU event, such as arithmetic overflow, illegal instruction, page fault, protection violation, misaligned access, or a software trap. Interrupt commonly refers to an external event such as an I/O request or reset. Terminology can vary, so the safest exam answer states the distinction used in the course and then groups the causes.

Classification axis	First category	Second category
Timing	Synchronous: reproducible at the same instruction/code position.	Asynchronous: normally generated externally and not tied to one instruction.
Request	User-requested: deliberate trap/system call/breakpoint.	Coerced: outside the user program's control.
Masking	Maskable: software can temporarily disable response.	Nonmaskable: must be serviced.
Position	Within an instruction: generated by that instruction.	Between instructions: observed at an instruction boundary.
Continuation	Resume after handler.	Terminate the program/system activity.

Typical causes grouped for an exam

Group	Examples
External / asynchronous	I/O request, reset, hardware malfunction, power failure.
Software requested	System call/trap, tracing, breakpoint.
Arithmetic	Integer overflow/underflow, floating-point anomaly.
Memory / protection	Page fault, misaligned access, memory-protection violation.
Instruction / hardware	Undefined or illegal opcode, hardware malfunction.

7.2 Restartable machines

A restartable machine can stop on an exception, save sufficient state, execute a handler, and restart without changing the intended program result. In a pipeline, several instructions are active simultaneously, so the machine must prevent the faulty instruction and younger instructions from modifying state after the exception is recognized. The handler needs the PC of the faulting or restart point, plus architectural status.

The lecture describes forcing a trap into the instruction stream, disabling all writes by the faulty and following instructions, saving the faulty instruction PC when the handler receives control, and using a special return-from-exception instruction to restore the PC and resume.

7.3 Precise exceptions

DEFINITION TO MEMORIZE

An exception is precise when the architectural state appears as if all instructions before the faulting instruction completed and no instruction after it completed. The faulting instruction and younger instructions can therefore be restarted from a well-defined point.

Precision is essential because the operating system and debugger need one coherent boundary in program order. If younger instructions have modified state while an older instruction remains incomplete, the handler cannot simply resume: it may repeat changes, lose changes, or observe a state that no sequential execution could produce.

In a simple in-order pipeline, one method associates an exception/status flag with every active instruction. When an exception is detected, the flag is set and that instruction is prevented from writing. The instruction continues toward the last stage. Only when it becomes the oldest instruction and reaches the final point is the exception taken. Younger instructions are suppressed or flushed. This restores program-order exception handling even when different stages detect different exceptions in the same cycle.

7.4 Multiple simultaneous and out-of-order exceptions

Different instructions can detect exceptions in the same cycle, for example an older load causing a data page fault in MEM and a younger arithmetic instruction causing an overflow in EX. The older instruction's exception must be handled first. It is also possible for a younger instruction to detect its exception earlier in time than an older one. Detection time must not decide exception order; program order must.

Per-instruction status carried down the pipeline solves this by recording the event without immediately changing control. When the flagged instruction reaches the retirement boundary, its exception is triggered. If an older exception flushes it first, its recorded exception never becomes architectural.

7.5 Why multicycle and out-of-order execution complicate precision

With different execution times, younger short instructions can finish before an older divide. If a younger instruction raises an exception after other younger instructions have already written results, or while an older instruction is unfinished, the visible state is imprecise. The lecture lists several approaches: accept imprecision; offer a slower precise mode; detect possible FP exceptions early and delay following instructions; or buffer results until all older instructions complete.

Buffering is the general solution developed later as the Reorder Buffer. Results can be calculated and forwarded out of order, but no register or memory update becomes architectural until the instruction commits in order. Exceptions are stored with the corresponding ROB entry and handled only when that entry reaches the head.

Organization	How precise order is obtained
Simple in-order pipeline	Carry an exception flag with each instruction, suppress its writes, and take the oldest flagged exception at the retirement/end point.
Multicycle in-order completion control	Prevent younger results from becoming visible before older uncertain operations, or buffer results.
Dynamic scheduling + speculation	Record results and exception status in the ROB; commit in program order; flush younger instructions when the faulting entry reaches the head.

7.6 ISA features that make restart harder

An instruction that changes state before it commits, such as an addressing mode that auto-increments a register, must be able to roll back that change when aborted. Implicit condition-code updates add state that must be tracked, saved, restored, and considered by scheduling. Complex variable-length or microcoded instructions are also harder to place cleanly in a regular pipeline. These complications are another reason simple, explicit ISA behavior is pipeline-friendly.

7.7 Full exam-ready answer

MODEL ANSWER

Exceptions are events that change normal control flow. Internal/synchronous examples include arithmetic anomalies, illegal instructions, page faults, misalignment, protection faults, traps, and breakpoints; external/asynchronous examples include I/O requests and reset. A precise exception presents a state in which every instruction before the faulting one has completed and no following instruction has completed, so execution can be safely restarted. In a simple pipeline, exception information is attached to each instruction, all writes by the faulting and younger instructions are disabled, and the oldest flagged instruction triggers the handler at the final stage. With out-of-order speculative execution, results and exception status are held in a ROB and architectural state is updated only by in-order commit. When the faulting instruction reaches the ROB head, the exception is handled and younger entries are flushed. This guarantees restartable, precise behavior.

Lecture grounding: 06 - exceptions.pdf, slides 2-26; 11 - hardware-based speculation.pdf, slides 5-24 for ROB-based precision. Past-paper emphasis: supplied 18 February 2026 precise-exception prompt.

Chapter 8 - Instruction-level parallelism and static techniques

EXAM PRIORITY: HIGH

Identify independent work, preserve correctness, and widen the scheduling window.

8.1 What ILP means

Instruction-Level Parallelism is the amount of overlap available among instructions. A pipeline exploits ILP by processing different stages of several instructions; multiple-issue and out-of-order processors try to exploit even more. Performance improves only when instructions are independent enough and the hardware or compiler can find and schedule them.

There are two broad approaches. Static scheduling relies on the compiler to rearrange instructions, rename registers, unroll loops, and form issue packets. Dynamic scheduling relies on run-time hardware to track operand availability and let ready instructions proceed. They can be combined: a compiler prepares good code while hardware adapts to cache misses, actual addresses, and branch outcomes.

Approach	Who finds/schedules parallelism	Strengths	Limitations
Static	Compiler before execution.	Simpler hardware; can perform global optimization and code transformation.	Cannot know every run-time address/delay; binary tied more closely to latency assumptions.
Dynamic	Processor during execution.	Adapts to actual dependencies and unpredictable delays; same binary across implementations.	Complex hardware, power, area, and verification cost.

8.2 Basic blocks and rescheduling

A basic block is a straight-line instruction sequence with no branch into the middle and no branch out except at the exit. Within it, the compiler can move independent instructions to fill latency gaps, provided data flow and exception behavior are preserved. Typical RISC basic blocks are only a few instructions long and often contain dependencies, so their available ILP is limited.

Static rescheduling preserves dependencies but changes issue order

```
# Original: immediate load-use stalls
lw  x1, 0(x10)
lw  x2, 0(x11)
add x5, x1, x2
sw  x5, 0(x14)
lw  x3, 0(x12)
lw  x4, 0(x13)
add x6, x3, x4
sw  x6, 0(x15)

# Scheduled: independent loads fill the gaps
lw  x1, 0(x10)
lw  x2, 0(x11)
lw  x3, 0(x12)
add x5, x1, x2
lw  x4, 0(x13)
sw  x5, 0(x14)
add x6, x3, x4
sw  x6, 0(x15)
```

8.3 Loop-level parallelism and loop unrolling

When different loop iterations are independent, their operations can overlap. Loop unrolling replicates the loop body several times and reduces the number of loop-control operations. An unroll factor of four processes four original iterations per transformed iteration, adjusting indices/pointers and handling any remainder when the trip count is not a multiple of four.

Unrolling has two main performance benefits in the lecture. It reduces branch and index-update overhead per data element. It also creates a larger scheduling region, allowing independent operations from different iterations to be interleaved around load and FP latencies. Costs include larger code, more register pressure, more complex cleanup code, and potentially poorer instruction-cache behavior.

```
# Original
for (i = 0; i < N; i++)
    x[i] = x[i] + y[i];

# Unrolled by 4
for (i = 0; i + 3 < N; i += 4) {
    x[i]   = x[i]   + y[i];
    x[i+1] = x[i+1] + y[i+1];
    x[i+2] = x[i+2] + y[i+2];
    x[i+3] = x[i+3] + y[i+3];
}
# A cleanup loop handles remaining elements if N is not a multiple of 4.
```

EXAM-READY ANSWER: LOOP UNROLLING

Loop unrolling is a mainly static compiler technique that reduces the number of loop iterations by replicating the body and updating the index by a larger step. It lowers loop-control and branch overhead and enlarges the body, exposing independent operations from different iterations for scheduling. Its disadvantages are increased code size, greater register pressure, possible instruction-cache cost, and the need to handle a remainder when the iteration count is not divisible by the unroll factor.

8.4 SIMD

Single Instruction, Multiple Data applies one operation to several data elements. Vector processors use vector instructions such as vector load, vector add, and vector store. GPUs contain many execution lanes that perform similar operations across large regular data sets. SIMD exploits data-level parallelism rather than discovering many unrelated scalar instructions one at a time.

8.5 Memory dependencies and aliasing

Register dependencies are easy to identify because the names are explicit. Memory dependencies are harder because different address expressions may refer to the same cell. A compiler that cannot prove two addresses are distinct must be conservative and preserve their order. Hardware can perform dynamic memory disambiguation after effective addresses are known, allowing more aggressive reordering when the addresses differ.

8.6 Register renaming

Register renaming replaces an architectural register name with a different temporary or physical name so unrelated values no longer conflict. It removes antidependencies and output dependencies, and therefore prevents register WAR and WAW hazards. It does not remove a true RAW relation: a consumer still needs the actual value produced by its predecessor.

Static renaming is performed by the compiler using spare architectural registers. Dynamic renaming is performed by hardware using reservation-station or ROB/physical-register tags. More names allow overlapping loop iterations without falsely tying every iteration to the same small set of registers.

8.7 Moving across branches: exception behavior and data flow

Preserving control dependencies is sufficient for correctness, but the deeper requirements are preserving exception behavior and data flow. Moving a potentially faulting load before a branch can create an exception on a path where the load should not execute, so it is unsafe unless the architecture suppresses or defers that exception. Moving an instruction that changes a value used after the merge can also change which definition reaches the consumer.

A safe speculative move must not create an observable exception on the wrong path and must not alter any value used by that path. Hardware speculation later achieves this by delaying architectural updates and exception handling until commit.

EXAM-READY ANSWER: ILP

ILP is the possibility of executing or overlapping independent instructions. Static techniques rely on the compiler; dynamic techniques rely on hardware. Inside a basic block the compiler reschedules independent instructions to fill latency slots, but basic blocks are short. Loop unrolling exposes parallelism across iterations and reduces control overhead, at the cost of code size and registers. Correct scheduling must preserve true data dependencies, memory order when addresses may alias, exception behavior, and the data flow reaching later instructions. Register renaming removes name dependencies but cannot remove true data flow.

Lecture grounding: 07 - ilp.pdf, slides 2-48; 02 - introduction to isa.pdf, slides 56-60 for compiler interaction. Past-paper emphasis: ILP/static-dynamic scheduling and loop-unrolling prompts supplied from 2020 and 2022.

Chapter 9 - Cache memories and the memory hierarchy

EXAM PRIORITY: HIGH

Master the address split, placement rule, replacement state, and write policy.

9.1 Why a cache is necessary

Processor execution is much faster than main-memory access. A cache is a small, fast memory placed between the processor and the larger, slower main memory. It keeps copies of recently used memory blocks so that most requests can be served with a shorter access time. The usefulness of a cache comes from locality.

Temporal locality means that an item accessed now is likely to be accessed again soon. Spatial locality means that addresses close to a recently accessed address are likely to be used soon. A cache therefore transfers and stores a complete block, not only the single requested byte or word.

A cache access is a hit when the requested block is already present and valid. It is a miss when the block is absent; the block must then be obtained from the next level, possibly after selecting a victim and writing back dirty data. The miss penalty is the extra time required to service a miss beyond the normal hit path.

PLAIN ENGLISH

The cache is the processor's small desk. Frequently used information stays on the desk; everything else is in the larger filing cabinet. Locality is the reason the desk works: programs often reuse the same data and nearby data.

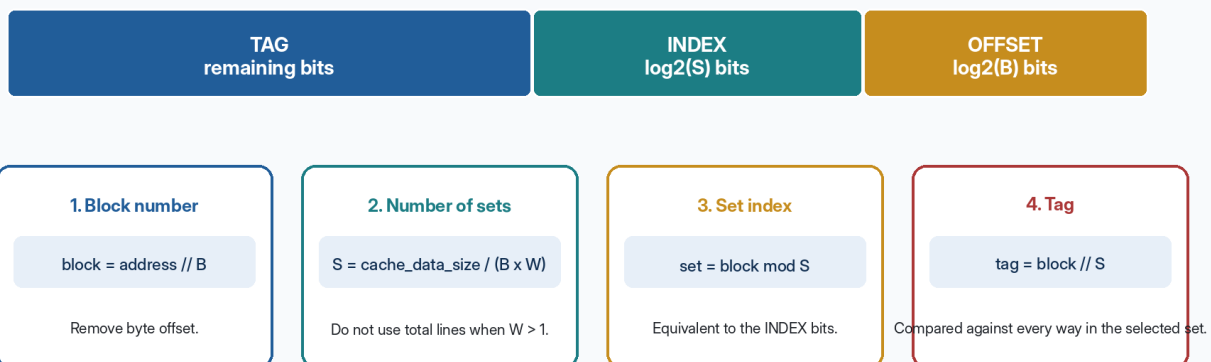
9.2 Cache organization: data array and directory

The data array contains cache blocks. The directory contains information used to determine which memory block, if any, is stored in each line. At minimum a line has a tag and a valid bit. A write-back cache additionally needs a dirty bit. Replacement algorithms may need age, recency, frequency, or pointer state.

A processor address is divided into three conceptual fields. The block offset selects a byte within the block. The set index selects the set that may contain the block. The tag distinguishes among all memory blocks that map to that same set. On a lookup, the indexed set is read, the requested tag is compared in parallel with the tag of every way, and a valid matching way signals a hit. A multiplexer returns the requested bytes from that way.

Cache address decomposition and mapping recipe

For a W-way set-associative cache with byte addresses



On a hit: valid bit = 1 and tag matches. On a miss: choose an invalid line first; otherwise apply LRU/FIFO/random and handle a dirty victim before replacement.

Address interpretation and set-associative cache lookup.

Field / state	Purpose
Block offset	Selects a byte or word inside the fetched block.
Set index	Selects the one set in which the block is allowed to reside.

Field / state	Purpose
Tag	Identifies which of the many blocks mapping to that set is actually stored.
Valid bit	Distinguishes a meaningful line from an unused or invalid line.
Dirty bit	In a write-back cache, indicates that memory is older than the cached copy.
Replacement state	Helps select a victim on a miss when all ways are valid.

9.3 The complete address-calculation formula

FORMULA SHEET

Let A be the address width in bits, C the cache data capacity in bytes, B the block size in bytes, and W the associativity. Number of lines $N = C/B$. Number of sets $S = N/W = C/(B \times W)$. Offset bits $b = \log_2(B)$. Index bits $s = \log_2(S)$. Tag bits $t = A - s - b$. For a numerical byte address: block number = address // B ; offset = address mod B ; set = block number mod S ; tag = block number // S .

These equations assume powers of two, as in the course exercises. Be careful about the unit of the given access. If the exercise already gives a block number, do not divide it by the block size again. In that case the set is simply block mod number of sets and the tag is block divided by number of sets.

The instruction alignment rule that appears in branch-predictor exercises is conceptually similar to the cache offset: low address bits that only select a byte within an aligned object do not identify a different instruction or block.

MOST COMMON CACHE-EXERCISE ERROR

Confusing byte address and block number. Write the unit beside every number before computing anything. BYTE ADDRESS -> divide by B first. BLOCK NUMBER -> start directly with modulo by the number of sets.

9.4 Placement alternatives

Organization	Where a block may be placed	Lookup cost	Miss/replacement behavior
Direct mapped ($W=1$)	Exactly one line: set = block mod N .	One tag comparison; simple and fast.	No victim choice, but conflict misses can be frequent.
W -way set associative	Any of W lines inside one indexed set.	W parallel tag comparisons and a way multiplexer.	Replacement is local to the selected set; fewer conflicts.
Fully associative	Any cache line; effectively one set with N ways.	All tags must be searched, usually with associative hardware.	Fewest placement conflicts but highest lookup cost.

Greater associativity gives each block more placement choices and normally reduces conflict misses. It also increases tag-comparison hardware, way-selection delay, energy, and replacement complexity. A direct-mapped cache can be viewed as one-way set associative; a fully associative cache can be viewed as one set whose associativity equals the number of lines.

9.5 Miss classes

Miss class	Reason	What usually helps
Compulsory (cold)	The block has never previously been brought into the cache.	Larger blocks or prefetching can exploit spatial locality.
Capacity	The active working set does not fit even with ideal placement.	A larger cache, better locality, or blocking the algorithm.
Conflict	Blocks compete for the same set even though unused space may exist elsewhere.	Higher associativity or changed data/layout/access pattern.
Coherence	Another core invalidates or changes a shared line.	Protocol/communication improvements; often treated separately from the classic 3C model.

DO NOT MIX CONCEPTS

Placement policy determines where a block may go. Replacement policy selects which allowed resident block leaves. Write policy determines how stores update the hierarchy. These are three different design decisions.

9.6 Replacement policies

Replacement is needed on a miss only when every way in the selected set is valid. Least Recently Used (LRU) evicts the line unused for the longest time. First In, First Out (FIFO) evicts the oldest inserted line regardless of later hits. Least Frequently Used (LFU) evicts the line with the lowest access count and needs a tie-breaking age rule. Random replacement uses little state but can discard a useful line. True LRU becomes expensive at high associativity, so real processors often use pseudo-LRU approximations.

In the supplied cache exercises, replacement metadata must be updated after every access exactly as the problem states. For an LRU convention where 0 means most recently used, the accessed/inserted line becomes 0 and the other lines in that set age consistently. For LFU, increment only the requested resident line on a hit, insert a new line with the stated initial count, and use the arrow/oldest rule when frequencies tie.

MECHANICAL LRU RULE FOR AN EXERCISE

Work on only the addressed set. On a hit, note the accessed line, then update all recency labels according to the exercise convention. On a miss, select the largest/oldest label as victim, insert the new block as MRU, and update the remaining labels. Never alter another set.

9.7 Write-through and write-back

Policy	On a write hit	Advantages	Costs / required state
Write-through	Update cache and the next level immediately.	Memory is kept current; simpler eviction and often simpler coherence reasoning.	High write traffic; commonly needs a write buffer.
Write-back	Update only the cache and mark the line dirty; write it to the next level on eviction.	Multiple writes to the same block generate less lower-level traffic.	Needs a dirty bit; a dirty victim increases miss service time.

A write miss needs a separate allocation policy. Write allocate fetches the block into the cache and then writes it, a natural partner for write-back because future writes can be absorbed locally. No-write-allocate, also called write around, sends the write to the next level without fetching the block and is often paired with write-through. The lecture emphasizes write-through versus write-back; mention allocation only when requested or when explaining a complete design.

9.8 Multilevel caches, inclusion, and coherence

Modern systems use a hierarchy. L1 is smallest and fastest and is often split into instruction and data caches. L2 is larger and slower. L3 may be shared by several cores. A hit at an upper level avoids accesses below it; a miss continues to the next level. Average Memory Access Time is commonly written as hit time plus miss rate times miss penalty, with the miss penalty itself including the lower-level hierarchy. The lecture also expresses a two-level weighted average as $p \times T_{\text{cache}} + (1-p) \times T_{\text{memory}}$; state the definitions so the formulas are not mixed.

In a multicore processor, private caches may hold copies of the same memory block. A coherence protocol keeps those copies consistent by tracking states and communicating changes. Valid/dirty concepts extend to states such as Modified, Exclusive, Shared, and Invalid (MESI). Coherence concerns the visibility/order of accesses to one memory location; consistency is the broader rule for ordering accesses to different locations.

AMAT

$\text{AMAT} = L1 \text{ hit time} + L1 \text{ miss rate} \times L1 \text{ miss penalty}$. For an L2 below L1, $L1 \text{ miss penalty} = L2 \text{ hit time} + L2 \text{ miss rate} \times \text{main-memory penalty}$. Use local miss rate unless the exercise explicitly asks for global miss rate.

9.9 Worked configuration 1: 64 KiB direct-mapped cache, 4-byte blocks

Assume 32-bit byte addresses and that 64 KiB describes data capacity. The cache has $C/B = 65536/4 = 16384$ lines. A direct-mapped cache has the same number of sets. Four-byte blocks need 2 offset bits, 16384 sets need 14 index bits, and the tag uses $32 - 14 - 2 = 16$ bits. Every line stores four data bytes plus at least a 16-bit tag and a valid bit. Ignoring valid bits, the tag array contains 16384×16 bits = 32 KiB, so data plus tags occupy about 96 KiB. This is why physical cache size is larger than the advertised data capacity.

Quantity	Result
Data capacity C	64 KiB = 65,536 bytes
Block size B	4 bytes
Lines / sets	16,384
Offset bits	2
Index bits	14
Tag bits	16
Tag storage, excluding valid	32 KiB

9.10 Worked configuration 2: 4 KiB, 2-way, 32-byte blocks

The number of lines is $4096/32 = 128$. With two ways there are $128/2 = 64$ sets. A 32-byte block requires 5 offset bits; 64 sets require 6 index bits; with 32-bit addresses, the tag is $32 - 5 - 6 = 21$ bits. For address 0x00001234, block number = $0x1234 // 32 = 0x91$, offset = 0x14, set = $0x91 \bmod 64 = 0x11$ (17), and tag = $0x91 // 64 = 2$.

Quantity	Result
Lines	128
Sets	64
Ways	2
Offset/index/tag bits	5 / 6 / 21
Example 0x00001234	block 0x91; set 17; tag 2; offset 0x14

9.11 Exam-ready cache answer

EXAM-READY ANSWER

A cache is a small and fast memory exploiting temporal and spatial locality by keeping copies of main-memory blocks. Each line contains a data block and directory information such as tag and valid bit; write-back caches also use a dirty bit. A byte address is divided into tag, set index, and block offset. The index selects a set, all tags in that set are compared, and a valid match is a hit. Direct mapping permits one location, set associativity permits one of W ways in a set, and full associativity permits any line. On a miss, a replacement policy such as LRU selects a victim. Write-through updates the next level on every store, while write-back updates it only when a dirty line is evicted.

Lecture grounding: 08 - cache.pdf, slides 2-67. Exercise conventions cross-checked against supplied cache papers using LRU and LFU tables, including the 12 February 2025 and 15 September 2025 formats.

Chapter 10 - Branch prediction: static methods, BHT, BTB, and correlation

EXAM PRIORITY: VERY HIGH

A very frequent theory and table exercise: always explain both prediction direction and target.

10.1 Why branches damage a pipeline

The fetch unit normally needs a next program counter every cycle. For a conditional branch, two facts may be unknown: whether the branch is taken and, if taken, the target address. Waiting until the branch executes creates idle cycles. Fetching a guessed path keeps the pipeline busy, but a wrong guess requires younger wrong-path instructions to be flushed. The performance cost therefore grows with branch frequency, misprediction rate, and the number of cycles between prediction and resolution.

Direction prediction answers Taken or Not Taken. Target prediction supplies the address used when the branch is predicted taken. A Branch History Table mainly predicts direction. A Branch Target Buffer mainly predicts the target early and may imply that a matching entry should be treated as taken, depending on the course model.

PLAIN ENGLISH

A branch is a fork in the road. The processor cannot stop at every fork, so it guesses a road and continues. A BHT guesses which road; a BTB remembers where the taken road leads.

PERFORMANCE RELATIONSHIP

Approximate branch CPI cost = branch frequency x misprediction rate x misprediction penalty. Overall CPI is the base CPI plus this and the other stall contributions.

10.2 Static prediction

Static prediction is chosen without adapting to the current run. Always-not-taken continues at PC plus the instruction length and works when branches are frequently not taken. Always-taken immediately chooses the target when it is available and often works for loops. Direction-based prediction commonly predicts backward branches taken and forward branches not taken. Profile-based prediction uses outcomes measured during representative earlier runs. Static mechanisms need little processor state but cannot adapt to input-dependent or phase-changing behavior.

Method	Rule	Strength	Weakness
Always not taken	Predict sequential PC.	No direction table; simple.	Poor for loop-closing branches.
Always taken	Predict branch target.	Good when most branches are taken.	Needs target early; poor for mostly-NT branches.
Direction based	Backward T, forward NT.	Captures common loop/if pattern statically.	Only a heuristic.
Profile based	Use behavior measured on training inputs.	Can specialize each branch.	May not match future inputs; compiler/profile workflow required.

10.3 General BHT architecture and behavior

A Branch History Table is a memory with N entries, each containing m prediction bits, commonly one bit or a two-bit saturating counter. When a conditional branch is encountered, low-order bits of its instruction address select an entry. Bits that only express instruction alignment are ignored. For fixed four-byte instructions, bits 0 and 1 are zero; an N-entry power-of-two BHT uses the next $\log_2(N)$ address bits.

The selected entry produces a taken/not-taken prediction. Fetch follows the predicted path. When the branch executes and its real outcome is known, the same entry is updated. Since a simple BHT does not necessarily store the full branch address, multiple branches can select the same entry. This aliasing means that one branch may overwrite or influence the prediction used by another.

BHT INDEXING FORMULA

For N power-of-two entries and fixed instruction size I bytes: $\text{index} = (\text{PC} \gg \log_2(I)) \& (N - 1)$. Example: 16 entries and 4-byte instructions use PC bits [5:2]. For 8 entries they use bits [4:2].

EXAM-READY ANSWER: GENERAL BHT

A BHT is a table of N entries, each containing an m-bit prediction state. When a conditional branch is decoded or fetched, depending on the implementation, the processor indexes the table with $\log_2(N)$ low-order PC bits after excluding instruction-alignment bits. The selected state predicts Taken or Not Taken. When the actual outcome becomes known, the addressed entry is updated. Because a basic BHT omits a full tag, different branch PCs can alias to the same entry and interfere.

10.4 One-bit BHT

A one-bit predictor stores the branch's most recently observed outcome: 0 predicts Not Taken and 1 predicts Taken. After resolution, the bit becomes the actual outcome. It learns quickly but changes its belief after a single exceptional result. For a loop branch that is taken repeatedly and not taken once at loop exit, a one-bit predictor can mispredict the exit and then mispredict the first taken branch the next time the loop starts, assuming no aliasing changes the bit between invocations.

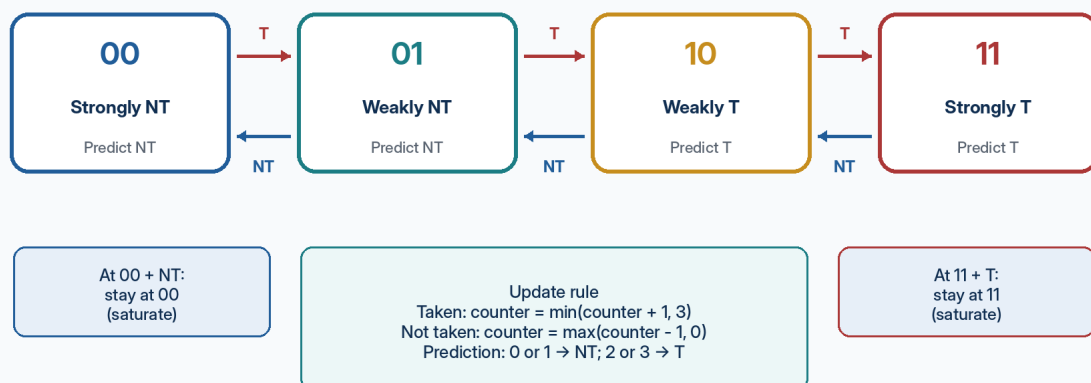
Old state	Prediction	Actual outcome	New state
0	NT	NT	0
0	NT	T	1
1	T	NT	0
1	T	T	1

10.5 Two-bit saturating-counter BHT

A two-bit counter supplies hysteresis: one unusual outcome is usually insufficient to reverse a strongly held prediction. States 0 and 1 predict Not Taken; states 2 and 3 predict Taken. On a taken outcome, increment unless already at 3. On a not-taken outcome, decrement unless already at 0. Saturation means the counter never wraps around.

2-bit saturating-counter predictor

The prediction changes side only after two consecutive contrary outcomes.



Two-bit saturating branch-predictor state machine.

State	Common name	Prediction	On NT	On T
00 / 0	Strongly not taken	NT	00	01

State	Common name	Prediction	On NT	On T
01 / 1	Weakly not taken	NT	00	10
10 / 2	Weakly taken	T	01	11
11 / 3	Strongly taken	T	10	11

WHY TWO BITS HELP

A normally taken loop reaches a taken state. Its single not-taken exit only weakens the counter, so the first taken iteration of the next invocation is still predicted taken. This removes the second loop-boundary error typical of a one-bit predictor.

10.6 Correlating predictors: the (m,n) idea

Some branches depend on the recent outcomes of other branches. An (m,n) correlating predictor keeps an m-bit global history register and uses n-bit prediction counters. The m history bits choose one of 2^m counters associated with the indexed branch entry. In the lecture's (2,2) example, each branch-table entry contains four two-bit counters. A two-bit shift register records recent outcomes, shifting in 1 for Taken and 0 for Not Taken. The current history chooses which of the four counters predicts and is updated.

Correlation can distinguish contexts that a simple per-address counter merges. Its costs are more storage and warm-up time, and destructive aliasing can still occur. The exact convention for which outcome becomes the most significant or least significant history bit must follow the exercise diagram.

EXAM-READY ANSWER: (2,2) PREDICTOR

A (2,2) correlating predictor combines two bits of global branch history with four two-bit saturating counters per indexed BHT entry. The PC selects the branch entry; the two-bit history selects one of its four counters. That counter predicts with states 0-1 = NT and 2-3 = T and is updated after resolution. The global history register is then shifted to include the actual result. It can exploit correlation between nearby branch outcomes, at the cost of additional table state and aliasing.

10.7 gselect and gshare

A global predictor must combine branch address information with global history. In gselect, some table-index bits come directly from the PC and the others from the global history register by concatenation. In gshare, PC bits and global history bits are exclusive-ORed to form the index. XOR tends to distribute patterns differently and can reduce some systematic aliasing. Both still use saturating counters in the pattern-history table.

10.8 Branch Target Buffer

A BTB is a small associative or indexed cache of branch PCs and their previously observed target addresses. It is accessed during instruction fetch because waiting until decode or execute would lose cycles. The current PC indexes or searches the BTB. If a valid entry's tag matches, the stored target is available as the candidate next PC. A miss normally continues sequential fetch. When a branch resolves, its entry can be allocated or updated with the actual target; the exact treatment of not-taken outcomes follows the predictor design stated by the exercise.

A full BTB entry may contain a valid bit, a branch-address tag, the target address, and optionally direction/type information. Storing a tag prevents an unrelated nonbranch or another PC that aliases to the same index from being treated as a known taken branch. In simplified course tables, an all-zero entry represents an empty BTB; a matching stored branch address and target represents a predicted-taken hit.

BTB item	Role
Index / associative lookup	Uses the current fetch PC to find a candidate entry.
Valid bit	States whether the entry is meaningful.
Branch PC tag/address	Confirms that the entry belongs to the current instruction.
Target address	Supplies the next PC immediately on a predicted-taken hit.
Optional direction/type state	Can cooperate with a BHT and distinguish branch classes.

BHT VERSUS BTB

BHT: should the branch be taken? BTB: where should fetch go if it is taken? A complete high-performance predictor often uses both. Never describe a target-only BTB as if it were merely a table of one-bit direction outcomes.

EXAM-READY ANSWER: BTB

A BTB is a cache-like structure accessed in the fetch stage with the current PC. Each valid entry stores enough branch-PC tag information to recognize the branch and a predicted target address. On a matching hit, the target can be sent immediately to the PC-selection logic, commonly implying a taken prediction or cooperating with a direction predictor. On a miss, fetch normally continues at the sequential PC. When the real branch and target are known, the corresponding entry is allocated or updated. Tags are important because simple indexing can cause collisions.

10.9 Mechanical procedure for a BHT table exercise

1. Copy the initial table before processing any branch. Do not update entries for nonbranch instructions.
2. Determine alignment bits. Four-byte instructions mean drop PC bits 1:0.
3. Compute index = $(PC \gg 2) \bmod N$, or read the specified PC bit slice.
4. Read the old state first and write the prediction from that old state.
5. Compare prediction with the actual result to mark correct or wrong.
6. Update exactly that entry after the comparison. For a one-bit table, write the actual outcome; for a two-bit table, saturating increment/decrement.
7. Process branches strictly in execution order because aliasing means an earlier branch can change a later branch's prediction.

Write this sequence beside every branch

```
Example: 8-entry, 1-bit BHT; four-byte instructions
branch PC = 0x00B20028
index = (0x00B20028 >> 2) & 0b111
       = low PC bits [4:2]
       = 2
old BHT[2] = 0 -> predict NT
actual = T -> misprediction
new BHT[2] = 1
```

10.10 Common branch-prediction traps

- Using target-address bits instead of branch-instruction address bits to index the BHT.
- Including byte-alignment bits in the index.
- Updating before recording the prediction; prediction must use the old state.
- Treating two-bit 01 as Taken. In the course convention, 0 and 1 predict NT; 2 and 3 predict T.
- Forgetting collisions: two different branch PCs may share one entry.
- Confusing a disabled branch delay slot with no branch penalty. Disabled means the architectural instruction after the branch is not guaranteed to execute; wrong-path work is still flushed/stalled.
- Giving a BHT answer when the question asks for a BTB with stored addresses and targets.

Lecture grounding: 09 - branch prediction.pdf, slides 2-40; 04 - pipeline.pdf for control hazards and delayed branching. Past-paper grounding includes the 2020 1-bit, two-bit, and (2,2) questions; 2021 and 2026 BHT tables; and 2026 BTB prompts.

Chapter 11 - Dynamic scheduling and Tomasulo's algorithm

EXAM PRIORITY: VERY HIGH

Know every structure, every field, and who reads or writes it at each stage.

11.1 Why dynamic scheduling exists

In an in-order pipeline, an instruction waiting for an operand or functional unit can block every younger instruction, even when some younger instruction is independent and ready. Dynamic scheduling moves part of instruction scheduling from the compiler into hardware. Instructions are still normally issued in program order, but after issue they may begin execution out of order when their operands and required unit are available.

This adapts to delays that the compiler cannot predict, such as cache misses and run-time memory addresses. It preserves binary portability across implementations with different latencies. The cost is substantial hardware for dependency tracking, operand buffering, arbitration, result broadcasting, recovery, and precise state.

Static scheduling	Dynamic scheduling
Compiler changes the order before execution.	Hardware chooses ready instructions at run time.
No run-time scheduling hardware is needed.	Requires queues, tags, wakeup/select logic, and recovery.
Limited by unknown memory aliases and unpredictable delays.	Can react to actual addresses, operands, and unit availability.
Code may be tuned to a particular latency model.	The same binary adapts to different implementations.

11.2 Dependency and hazard consequences of out-of-order execution

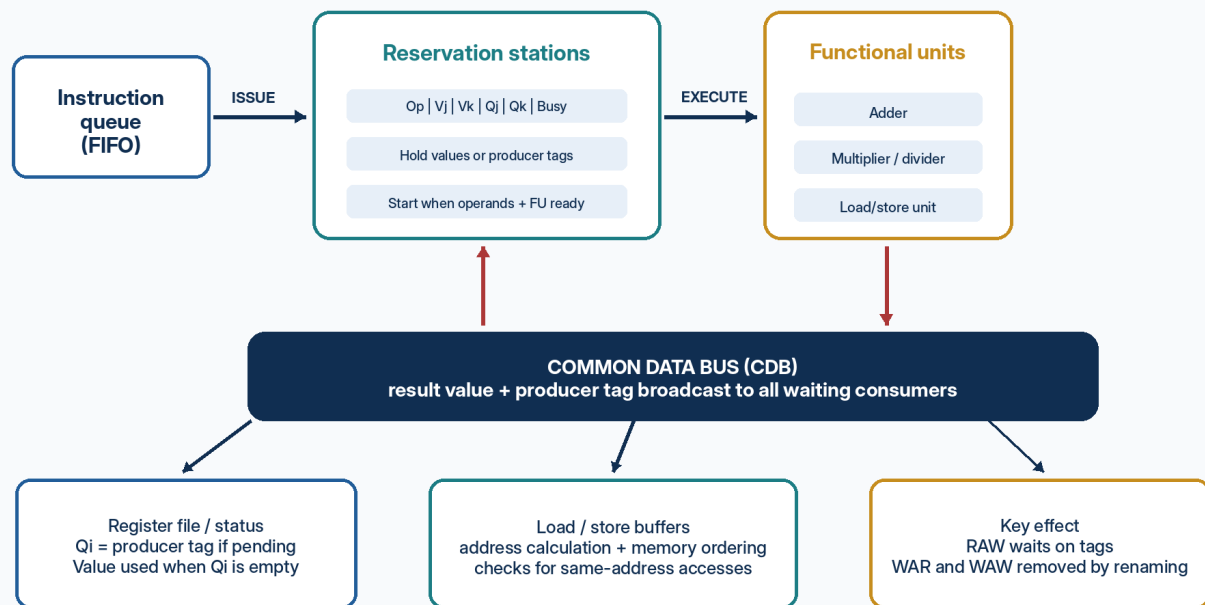
True RAW dependence must always be respected: a consumer cannot execute until its producer has generated the required value. Out-of-order execution can create WAR and WAW hazards because a younger instruction may read or write before an older instruction reaches the corresponding operation. Register renaming eliminates these name hazards by assigning each produced value a unique tag or physical name. Memory dependencies remain difficult because addresses may not be known at issue.

Execution out of order also makes architectural state appear out of program order unless completion is controlled. Basic Tomasulo solves dependency and name-hazard problems but, without an ROB, can still create imprecise exceptions because results are written when ready. Hardware speculation with an ROB, covered in Chapter 12, adds in-order commit.

11.3 Tomasulo architecture at a glance

Tomasulo dynamic scheduling: data and tags move through the machine

Issue is in order; execution and result production may be out of order.



Conceptual Tomasulo datapath: issue, reservation stations, functional units, and result broadcast.

The instruction queue supplies operations in program order. Reservation stations sit in front of functional units and hold issued operations, ready operand values, or tags identifying future producers. Functional units execute ready operations. Load/store buffers hold memory operations and effective-address information. The Common Data Bus broadcasts completed values and producer tags to all interested structures. Register-result-status information tells the issue logic whether each architectural register is ready or will be produced by an outstanding instruction.

11.4 Reservation stations: purpose and fields

A reservation station decouples issue from execution. An instruction can leave the central issue path even when one or both operands are unavailable. It waits close to the appropriate functional unit, watching the CDB for missing values. When all operands are ready and a compatible functional unit is available, the scheduler selects it for execution.

Field	Meaning
Busy	Whether the reservation-station entry is occupied.
Op	Operation to perform, for example add, subtract, multiply, or divide.
Vj, Vk	Actual values of source operands that are already available.
Qj, Qk	Tags of the producer entries expected to generate missing operands. Zero/empty means the corresponding V field is ready.
A	Immediate/displacement/effective-address information, especially for load/store entries.
Destination tag	In a speculative design, often the ROB entry or physical destination associated with the result.

At issue, the instruction-decode/rename logic writes a free reservation-station entry. For each source, it copies the register value if ready or the current producer tag if not. The destination's register-status entry is changed to name this new producer, which is dynamic register renaming. During write-result, every occupied reservation station compares its Q tags with the tag broadcast on the CDB. A match captures the value into V and clears Q. At execution start, the functional-unit scheduler reads a ready station; when execution is complete and the result has been accepted for broadcast, the station can be released according to the implementation.

EXAM-READY ANSWER: RESERVATION STATIONS

Reservation stations are distributed instruction/operand buffers placed before functional units in Tomasulo's architecture. They let an issued instruction wait without blocking younger independent instructions. An entry contains Busy, Op, ready values Vj/Vk, missing-producer tags Qj/Qk, and address/immediate field A; a speculative implementation also associates a destination/ROB tag. Issue writes the entry from the instruction and register-status information. The CDB writes arriving operands when its tag matches Qj or Qk. A functional unit reads a ready entry when both operands are available, and the entry is released after execution/result handling. Tags implement dynamic register renaming and remove WAR/WAW name hazards.

11.5 Register result status / renaming table

For each architectural register, the status table records either that the register file holds the current value or the tag of the youngest issued instruction that will define it. When a new writer issues, its station or destination tag replaces the old association. A later reader takes the value if the entry is clear; otherwise it takes the producer tag. When a result broadcasts, the register file is updated only if the table still points to that producer. This last condition prevents an older result from overwriting the name of a younger pending definition.

Why the tag comparison on write-back matters

```
I1: MUL F2, F4, F6    -> register-status[F2] = Mul1
I2: ADD F2, F8, F10   -> register-status[F2] = Add1
```

If Mul1 finishes first, its value may wake its real consumers, but it must not clear/write F2 as the current mapping because F2 now names Add1. This is how WAW is removed by renaming.

11.6 The Common Data Bus

The CDB is the shared result-broadcast network connecting completed functional units and memory-load results to waiting consumers. A broadcast carries a producer tag and its value. Reservation stations compare the tag against Qj/Qk and capture matching operands. The register file or rename/status structure observes it. In a speculative Tomasulo machine, the ROB also receives the value and marks the producing entry ready.

Data are placed on the CDB as soon as an instruction has completed execution and wins access to the bus. With one CDB, only one result can be broadcast per cycle. If several functional units finish together, all but one must wait even though their calculations are complete. Multiple CDBs reduce this write-result bottleneck and can wake more dependents in one cycle, but require more wiring, ports, comparators, arbitration, area, and power.

CDB writers	CDB readers
Functional units whose result is complete.	All reservation stations/load-store structures waiting for a matching tag.
Load/memory unit when a loaded value is available.	Register file or register-status/rename structure.
Other completed producer units allowed by the implementation.	ROB in a speculative implementation.

EXAM-READY ANSWER: CDB

The Common Data Bus is Tomasulo's broadcast path between result-producing units and all structures waiting for values. Functional units and the load/memory unit write a tag-value pair when an instruction finishes and obtains bus arbitration. Reservation stations read it to replace matching Qj/Qk tags with Vj/Vk values; the register file/rename state and, with speculation, the ROB also read it. A single CDB is a bottleneck when several units finish in the same cycle because only one result can be broadcast. Multiple CDBs improve completion and wake-up bandwidth, at the cost of extra ports, comparators, wiring, control, area, and power.

11.7 Tomasulo stages

Stage 1 - Issue and rename

The next instruction is considered in program order. It can issue only if a compatible reservation station or load/store buffer is free; in a speculative machine a free ROB entry is also required. Source operands are copied as values if ready

or as producer tags if pending. The instruction's destination is renamed to the new producing tag. If no required entry is available, issue stalls and younger instructions cannot bypass this point in the basic scheme.

Stage 2 - Wait and execute

The instruction waits until Qj and Qk are empty and any other conditions are satisfied. A ready instruction may begin when its functional unit is available, even if older instructions are still waiting, so execution can be out of order. A pipelined functional unit may accept a new operation every initiation interval; an unpipelined unit remains occupied for its full latency.

Stage 3 - Write result

After the operation completes, it waits for CDB access if necessary, then broadcasts its tag and value. Waiting stations capture matching operands. The architectural register is updated only if this producer is still its current mapping in non-ROB Tomasulo. The producer's station is released. With an ROB, the value instead becomes ready in the ROB and architectural update waits for commit.

THREE WORDS TO REMEMBER

ISSUE: allocate + rename. EXECUTE: wait for operands/unit, then calculate. WRITE RESULT: arbitrate + broadcast + wake consumers. Speculative Tomasulo adds a fourth logical stage: COMMIT in program order.

11.8 Loads, stores, and memory disambiguation

A load/store operation first calculates an effective address. A load can read memory only when doing so will not violate an older store to the same address. A store has both an address and a data operand and must preserve visible memory order. If an older store's address is unknown, a conservative machine holds a younger load because they might alias. More aggressive hardware predicts independence or compares addresses when known and forwards data from an older store to a matching younger load.

Basic Tomasulo examples often use load and store buffers and simplified ordering rules. In an exam snapshot, follow the exact lecture model: distinguish address calculation from memory access and do not let a store update memory before its required operands and ordering conditions are satisfied. With an ROB, stores update memory only at commit.

11.9 Full worked dependency example

How independent work bypasses a wait

```
I1: LD    F2, 0(R1)
I2: MUL.D F4, F2, F6
I3: ADD.D F8, F10, F12
I4: SUB.D F14, F4, F8
```

I1 issues to a load buffer and renames F2 to its load tag. I2 issues to a multiply station: F6 is copied as a value, while Qj receives I1's tag because F2 is pending. I3 issues to an add station with both operands ready and can execute before I2. I4 takes tags for both I2 and I3. When I3 broadcasts, I4 captures one operand. When I1 broadcasts, I2 captures F2 and begins multiplication. When I2 broadcasts, I4 becomes fully ready and can execute. The true RAW chain remains, but unrelated I3 is not blocked behind it.

11.10 Scoreboarding versus Tomasulo

Aspect	Scoreboarding	Tomasulo
Operand waiting	Central scoreboard tracks hazards and unit status.	Reservation stations hold values/tags near FUs.
Register renaming	Classic scoreboard does not fully rename; WAR/WAW must be prevented by scheduling.	Tags rename destinations and remove register WAR/WAW.
Result distribution	Controlled write-result stage.	CDB broadcasts tag-value pairs.
Main idea	Centralized hazard control.	Distributed wakeup plus renaming.

11.11 Common Tomasulo mistakes

- Writing both a value and a producer tag for the same operand. V is used when ready; Q is used when pending.
- Assuming issue itself is out of order. In the basic lecture model, issue is in program order; execution is out of order.
- Starting execution when one Q field is still nonempty.
- Updating all registers on any broadcast. Only the destination whose current tag matches that producer is affected.
- Freeing a station merely when arithmetic ends even though the result still waits for the single CDB.
- Claiming renaming removes RAW. It removes WAR and WAW name hazards, not true data flow.
- Ignoring memory address conflicts or allowing a speculative store to modify memory early.

Lecture grounding: 10 - dynamic scheduling.pdf, slides 2-50. Architectural connections continued in 11 - hardware-based speculation.pdf. Past-paper grounding: CDB questions from 2021 and 2026 and the 2026 reservation-station prompt.

Chapter 12 - Hardware speculation, Reorder Buffer, and precise commitment

EXAM PRIORITY: VERY HIGH

The ROB is the bridge between out-of-order execution and an in-order architectural state.

12.1 Why dynamic scheduling alone is not enough

Dynamic scheduling lets independent operations execute around delays. Branch prediction lets the processor fetch beyond an unresolved branch. Speculation combines them by executing predicted-path instructions before it is certain that they should affect the program. The danger is that wrong-path values, stores, or exceptions could become architecturally visible. A Reorder Buffer prevents this by holding speculative results until instructions reach the head in program order.

The resulting machine can issue and execute out of order and can write completed values into temporary structures out of order, but it commits architectural effects in order. This separation is the central idea to state in every ROB answer.

PLAIN ENGLISH

The processor may do homework questions in any convenient order, but it submits the answers in the original order. The ROB is the ordered submission queue. A wrong branch lets it throw away unsubmitted work.

12.2 ROB organization

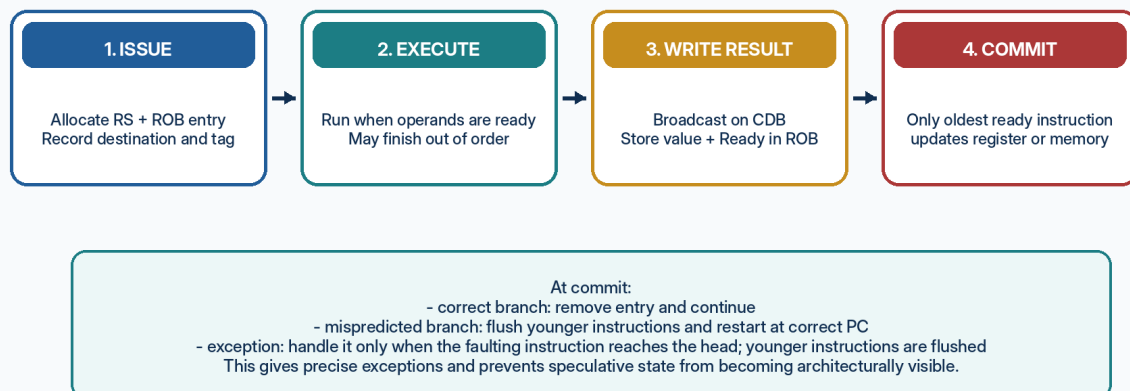
The ROB is a FIFO implemented as a circular buffer with head and tail pointers. Issue allocates at the tail; commit examines the head. Each outstanding instruction has an entry. The exact implementation can contain more fields, but the lecture and supplied answer guidelines emphasize instruction type, destination, value, and ready/done state. Practical entries also store exception information, branch prediction/recovery information, and sometimes store address/data.

ROB field	Meaning
Busy / valid	The entry is allocated to an in-flight instruction.
Instruction type	Register operation, load, store, branch, and so on; controls commit behavior.
Destination	Architectural register or memory destination to update at commit.
Value	Completed result held before architectural update.
Ready / done	Execution has completed and the value/status is available.
Exception / branch status	Records a deferred exception or whether a prediction was correct.
Store address/data	Allows a store to wait until it is oldest before updating memory.

12.3 ROB lifecycle

Speculation with a Reorder Buffer (ROB)

Compute out of order, but update architectural state in program order.



The four-stage lifecycle of an instruction in a speculative Tomasulo processor.

1. Issue / allocation

An instruction issues only when a suitable reservation station and a free ROB entry are available. The tail entry is allocated and marked busy/not ready. The destination register is renamed to that ROB entry, so consumers can refer to the value before it commits. The instruction is placed in a reservation station with source values or source ROB tags.

2. Execute

When operands are ready, the instruction executes, possibly before older instructions. A load performs permitted memory access; a branch computes its real outcome and target; a store computes its address and data but does not yet change architectural memory.

3. Write result

The functional unit broadcasts the value and ROB tag on the CDB. Waiting reservation stations capture it. The matching ROB entry stores the value and is marked ready. A result may be forwarded from the ROB/CDB to consumers even though it has not committed.

4. Commit / retire

Only the oldest ROB entry is eligible. If it is not ready, commit waits even if younger entries are ready. A normal register result updates the architectural register only if appropriate for the current mapping. A store writes memory now. A correctly predicted branch retires. If the oldest instruction has a recorded exception, the processor takes it with a precise state. If a branch was mispredicted, younger speculative instructions are flushed and rename/PC state is restored. The head entry is then released.

12.4 Who reads and writes the ROB

Event / module	ROB access
Issue/rename logic	Allocates the tail entry and writes type, destination, and initial not-ready status.
Functional unit via CDB	Writes completed value/status and marks the addressed entry ready.
Source-operand selection	Reads a ready ROB value or uses the ROB number as a pending tag.
Commit logic	Reads the head entry, applies its architectural effect, and deallocates it.
Branch recovery logic	Flushes all younger entries after a misprediction and restores mappings.
Exception logic	Records the exception in the producing entry; services it only when the entry reaches the head.

12.5 In-order commit and precise exceptions

An exception is precise when all older instructions have completed their architectural effects, the faulting instruction and all younger instructions have not changed architectural state, and the saved PC identifies the faulting/restart point. In-order commit creates exactly this boundary. Results can finish early, but until their entries become the oldest they remain speculative. An exception is stored in its ROB entry rather than immediately trapping. If it reaches the head, all preceding instructions have committed and all younger ones can be discarded.

A branch misprediction is handled similarly to an exception in that younger work is invalid. The branch itself reaches the point where its outcome is known; the machine redirects the PC and flushes younger ROB and reservation-station state. Older entries are preserved and may continue to commit.

PRECISE STATE TEST

At an exception boundary: every older instruction appears completed; the faulting instruction and every younger instruction appear not completed. The architectural register file and memory must match that sequential prefix.

12.6 Why stores must wait for commit

A register result can be held in a ROB entry without changing the architectural register file. A store is more dangerous because writing memory is difficult to undo and can be visible to devices or other processors. Therefore the speculative machine computes and buffers store address and data, but writes the cache/memory only when the store is at the head and ready. A younger load may sometimes receive data by store-to-load forwarding, yet the external architectural store still waits for commit.

12.7 RAT, ROB, and architectural register file

The Register Alias Table maps each architectural destination register to the youngest in-flight producer, often a ROB tag in the lecture model. A source reads the architectural register file when no pending mapping exists, reads a ready ROB value when its producer has completed, or carries the ROB tag to wait for the CDB. At commit, the register file receives the value. The mapping is cleared only if it still points to the committing entry; a younger definition may already own the architectural name.

Mapping test at commit

```
I1: ADD F2, F4, F6    -> RAT[F2] = ROB3
I2: MUL F2, F8, F10  -> RAT[F2] = ROB4
```

ROB3 may commit its sequential result to the architectural file, but RAT[F2] must remain ROB4 because the younger value is still pending. When ROB4 commits, it becomes the final architectural F2.

12.8 ROB capacity and performance limits

The ROB bounds the number of instructions that may be in flight and therefore the distance over which the processor can search for independent work. If the ROB is full, issue stalls even when reservation stations are available. Commit bandwidth can also be a bottleneck. A long-latency instruction at the head blocks retirement of every younger completed instruction, called head-of-line blocking. A larger ROB exposes more ILP but increases area, energy, tag widths, wakeup complexity, and recovery work.

12.9 Complete exam-ready ROB answer

EXAM-READY ANSWER

The Reorder Buffer is a circular FIFO that allows instructions to execute and produce temporary results out of order while committing architectural state in program order. A ROB entry is allocated at successful issue and typically stores instruction type, destination, value, ready state, and exception/branch information. The functional unit writes the result through the CDB and marks the entry ready; consumers may read or wait for that ROB tag before commitment. Commit examines only the oldest entry. A ready register result updates the architectural register, and a ready store updates memory; the entry is then deallocated. If the head contains an exception it is serviced there, and if a branch is mispredicted younger entries are flushed. Thus the ROB supports speculation, in-order commitment, recovery, and precise exceptions.

12.10 Common ROB mistakes

- Saying that completed instructions immediately update architectural state. They write the ROB/CDB first; commit is separate.
- Allowing a ready younger entry to commit around an unready head. Commitment is in order.
- Describing the ROB as the same thing as a reservation station. The RS waits/schedules near a functional unit; the ROB preserves program order and architectural recovery.
- Forgetting that stores write memory at commit, not at speculative execution completion.
- Taking an exception as soon as the functional unit detects it. The exception is recorded and handled when oldest for precision.
- Flushing older entries on a branch error. Only younger speculative work is removed.

Lecture grounding: 11 - hardware-based speculation.pdf, slides 2-24; 10 - dynamic scheduling.pdf for Tomasulo foundations; 06 - exceptions.pdf for precise state. Past-paper wording cross-checked against the 2021 and 2022 ROB answers and 2026 CDB/reservation-station prompts.

Chapter 13 - Multiple-issue and superscalar processors

EXAM PRIORITY: CORE

Understand how CPI can fall below one and why issue width alone does not guarantee speedup.

13.1 From pipelining to multiple issue

A scalar pipeline aims to complete approximately one instruction per cycle after filling. A multiple-issue processor can start more than one instruction in a cycle and, with enough parallelism and resources, achieve an average CPI below one. If the issue width is W , the ideal instruction throughput is W instructions per cycle and the ideal CPI lower bound is $1/W$. Real throughput is lower because instructions are dependent, branches interrupt the stream, caches miss, and resources or front-end bandwidth are limited.

Multiple issue requires the machine to fetch, decode, check, rename, dispatch, execute, write back, and retire several instructions per cycle. Increasing only the number of functional units is insufficient: every supporting structure must have adequate width and ports.

PLAIN ENGLISH

A normal pipeline is one checkout lane accepting roughly one customer each cycle. Multiple issue opens several lanes. It helps only when several independent customers are ready and the entrance, payment system, and exit can also handle them.

IDEAL LIMITS

Issue width $W \rightarrow$ peak IPC = W and ideal CPI = $1/W$. Actual IPC is bounded by available ILP, front-end bandwidth, functional units, memory behavior, branch accuracy, result bandwidth, and retirement width.

13.2 Static versus dynamic multiple issue

Feature	Static multiple issue	Dynamic multiple issue / superscalar
Who forms the issue group	Compiler schedules operations into legal packets.	Hardware examines fetched instructions each cycle.
Dependency checking	Mostly performed before execution by the compiler.	Performed dynamically with issue/rename/scheduling logic.
Response to cache/memory delay	Packet schedule cannot fully adapt at run time.	Independent operations can be selected around actual delays.
Hardware complexity	Lower scheduling complexity.	High complexity, power, and verification cost.
Binary dependence	More tied to resource/latency assumptions.	Same binary can run on different issue widths/latencies.

13.3 Static two-issue example

The lecture uses a representative packet containing one integer/memory/branch operation and one floating-point operation. The compiler must place only mutually legal operations in the same packet and insert an empty slot when no independent instruction is available. If a packet is encoded as a fixed group, both operations share issue timing; a hazard affecting one slot may delay the complete group, depending on the architecture.

A wider static schedule changes apparent delay distances. Suppose a load value becomes usable after a fixed number of cycles. Since two instructions may be issued each cycle, the compiler may need to place several independent instructions, not merely one, between the load and its consumer. Branch delay measured in cycles likewise corresponds to several instruction slots.

Illustrative two-issue packet schedule

Cycle / issue packet	Integer-memory slot	FP slot
1	lw x1, 0(x2)	fadd f4, f6, f8
2	addi x2, x2, 4	[empty]
3	sw x1, 0(x3)	fmul f10, f4, f12

KEY STATIC-SCHEDULING RULE

The compiler must check both dependencies within one packet and dependencies between packets. Empty slots preserve correctness but waste issue bandwidth and increase code size.

13.4 Dynamic superscalar front end

A dynamic superscalar processor fetches and decodes several instructions, but in the basic Tomasulo/ROB model it usually issues them in program order to preserve a simple recovery point. Each instruction needs a free destination-renaming resource, ROB entry, and suitable reservation station. If an older instruction cannot issue, younger members of the group may be blocked unless the front end contains additional buffering and more advanced dispatch rules.

After dispatch, wakeup/select hardware chooses ready instructions from a larger window and sends them to available functional units. This stage is out of order. Completed instructions write values to physical registers or result buses; retirement remains in order. Wide machines need multiported rename tables, register files, ROBs, caches, load/store queues, and bypass networks.

Pipeline area	What width creates
Fetch	Multiple instruction bytes per cycle, alignment across cache blocks, and multiple branch handling.
Decode/rename	Several decoders, intra-group dependency detection, multiple register mappings and ROB allocations.
Wakeup/select	Many tag comparisons and arbitration among ready instructions.
Execute	Several functional units and enough operand/read ports.
Write result	Several result ports or CDBs; complex bypass network.
Commit	Ability to retire several oldest ready instructions while preserving order and exceptions.

13.5 Intra-group dependencies

When several consecutive instructions are considered in one cycle, a younger instruction may depend on an older instruction in the same group. The rename stage must detect this chain and give the younger source the newly created producer tag, not the stale architectural mapping. Multiple writers to the same architectural register in one group must be ordered so that the youngest writer becomes the final mapping. These comparisons grow rapidly with width and are one reason very wide superscalar front ends are difficult to clock.

Dependencies inside one decoded group

```

I1: add x5, x1, x2
I2: sub x6, x5, x3    # I2 must receive I1's new tag
I3: mul x5, x7, x8    # final RAT[x5] must name I3, not I1

```

13.6 CDB and result-bandwidth bottleneck

Multiple issue creates multiple potential completions in one cycle. A single CDB permits only one tag-value broadcast, so completed operations queue for write result. This delays their dependent instructions and can keep reservation stations occupied. Multiple CDBs or a multiported physical register file increase bandwidth, but every waiting station then needs more tag comparators and the wiring becomes expensive. The professor has repeatedly used this point in CDB theory questions.

13.7 Why actual IPC is below issue width

- True RAW chains serialize instructions regardless of how many units exist.
- Name dependencies need renaming resources; memory aliases need load/store ordering.

- A mispredicted branch discards a larger amount of work in a wide/deep machine.
- Instruction-cache misses or difficult alignment can starve the entire back end.
- Functional-unit mix may not match the available operations.
- Load/store queues, cache ports, CDBs, or commit bandwidth may saturate.
- The finite ROB/instruction window may not reach far enough to find independent operations.
- Power and thermal limits may prevent all units from operating at peak simultaneously.

13.8 Superscalar versus superpipelined

Concept	How throughput is increased	Main consequence
Superscalar	Issue several instructions in the same cycle using parallel resources.	Wider control/data paths and need for more ILP.
Superpipelined	Split work into more, shorter stages and raise clock frequency.	Deeper pipeline, longer hazard/branch penalties in cycles.
Both combined	Wide issue plus deep stages.	High peak throughput but complex, power-hungry recovery and forwarding.

13.9 Exam-ready answer

EXAM-READY ANSWER

A multiple-issue processor can start more than one instruction per cycle, so an ideal width- W machine has peak IPC W and CPI $1/W$. Static multiple issue relies on the compiler to build legal issue packets, inserting empty slots when no independent operation fits. A dynamic superscalar processor uses hardware renaming, reservation stations, an instruction window, multiple functional units, and a ROB to find and execute ready instructions out of order while committing in order. Performance is limited by true dependencies, branch prediction, memory stalls, issue-window size, functional-unit mix, result/CDB bandwidth, and retirement width. Wider issue therefore increases potential throughput but also front-end, port, bypass, power, and recovery complexity.

Lecture grounding: 12 - multiple-issue processors.pdf, slides 2-20; 10 - dynamic scheduling.pdf and 11 - hardware-based speculation.pdf for the dynamic superscalar structures; 04 - pipeline.pdf and 05 - multicycle.pdf for latency/issue constraints.

Chapter 14 - VLIW and EPIC processors

EXAM PRIORITY: SUPPORTING

A compiler-scheduled alternative to dynamic superscalar issue.

14.1 VLIW principle

VLIW means Very Long Instruction Word. One long instruction contains several operation fields intended for different functional units in the same cycle. The compiler discovers independent operations, assigns them to slots, and guarantees that the packet is safe. Hardware can therefore dispatch the operations without performing the same complex run-time dependency checks as a superscalar processor.

The compiler must know the number and type of functional units, operation latencies, legal slot combinations, and memory behavior assumptions. Empty operation slots are encoded as no-ops or compressed by implementation-specific techniques. A whole packet often advances together, so a stall affecting one operation may hold all operations in that packet.

PLAIN ENGLISH

Superscalar hardware receives normal instructions and decides which can run together. VLIW asks the compiler to package operations that are already proven safe to run together.

Conceptual VLIW word

```
| Integer ALU | Memory | FP add | FP multiply |
| add r1,...  | load   | fadd  | [empty]    |
```

The four fields form one very long instruction word.
The compiler, not the issue hardware, guarantees independence.

14.2 Advantages

- Simpler issue hardware because dependency checking and operation packing are mainly performed by the compiler.
- Potentially lower control complexity, area, and power for a given number of functional units.
- Parallelism is explicit in the binary, making the intended schedule visible.
- Long compiler analysis can use a wider scope than a small run-time instruction window.
- Deterministic scheduling can be attractive in embedded or signal-processing workloads.

14.3 Disadvantages and limits

The instruction words are wide, and many slots may be empty when the code lacks the required mix of independent operations. This increases code size and instruction-fetch bandwidth. Scheduling across several cycles requires many live intermediate values and therefore high register pressure. Memory latency, cache misses, and branch outcomes are difficult to predict statically. The compiler may add compensation code or conservative waits, reducing efficiency.

Binary compatibility is a major issue. A schedule optimized for one number of units and one latency may be wrong or inefficient on another implementation. A dynamic superscalar binary expresses dependencies but leaves timing choices to the hardware; VLIW encodes more of the timing/resource schedule itself. Changing the implementation may therefore require recompilation.

Problem	Why it occurs
Code expansion	Every word reserves several slots; unused slots waste bits.
Register pressure	Many overlapping operations keep values live for longer.
Unpredictable latency	The compiler cannot know every cache miss or run-time conflict.
Branch handling	Large scheduled regions need speculation/predication and recovery code.
Binary compatibility	The packet assumes a particular resource and latency model.
Limited ILP	Real code may not contain enough independent operations of the right types.

14.4 Trace scheduling, predication, and speculation

A compiler can select a likely path through several basic blocks, called a trace, and schedule operations across branch boundaries. It must preserve data flow and exception behavior on every possible path. Predication converts some branches into conditionally executed operations, enlarging the scheduling region but executing operations whose results may be discarded. Compiler speculation may move safe operations before a branch; potentially faulting operations require architectural support to defer or suppress wrong-path exceptions.

14.5 EPIC

EPIC, Explicitly Parallel Instruction Computing, extends VLIW ideas with architectural mechanisms intended to make compiler-exposed parallelism more flexible. Instruction groups explicitly mark operations that can execute in parallel, while features such as predication, speculative loads, rotating registers, and dependency/check mechanisms support aggressive schedules. The basic comparison remains: EPIC/VLIW communicate parallelism from the compiler, whereas superscalar hardware discovers more of it dynamically.

14.6 VLIW versus superscalar

Dimension	VLIW / EPIC	Dynamic superscalar
Scheduler	Compiler.	Hardware at run time.
Binary format	Wide packets or explicit parallel groups.	Ordinary sequential instruction stream.
Run-time adaptation	Limited; relies on prepared alternatives/support.	Adapts to actual readiness and cache delays.
Hardware complexity	Lower issue/dependency-check complexity.	High rename, wakeup, select, and recovery complexity.
Compiler complexity	High; machine model and scheduling are central.	Still important, but hardware can recover missed scheduling opportunities.
Compatibility	Can depend strongly on FU/latency assumptions.	Normally better across implementations of the same ISA.

14.7 Exam-ready answer

EXAM-READY ANSWER

VLIW encodes several independent operations in one very long instruction, with each slot associated with a functional-unit class. The compiler performs dependency analysis, schedules operations, and inserts empty slots, so issue hardware is simpler than in a dynamic superscalar processor. Advantages are lower run-time scheduling complexity and explicit parallelism. Disadvantages are wide code, unused slots, high register pressure, limited adaptation to cache misses and branches, demanding compilers, and binary dependence on the assumed resource/latency model. EPIC retains explicit parallelism while adding predication and speculation support to improve scheduling flexibility.

Lecture grounding: 13 - VLIW processors.pdf, slides 2-26; 07 - ilp.pdf for compiler scheduling and loop transformations; 06 - exceptions.pdf for safe speculation requirements.

Chapter 15 - Thread-level parallelism and multithreading

EXAM PRIORITY: SUPPORTING

Know fine-grained, coarse-grained, SMT, and multicore distinctions.

15.1 From ILP to TLP

Instruction-level parallelism looks for independent operations within one instruction stream. Thread-level parallelism uses several independently scheduled instruction streams. A thread has its own program counter, architectural register state, and control flow, while threads may share an address space and parts of the memory hierarchy. Separate processes normally have stronger protection and separate address spaces, though the hardware scheduling ideas are related.

TLP is useful when one thread lacks enough ILP or stalls on memory. Instructions from another thread can use otherwise idle execution resources. The principal performance goal is normally higher throughput and utilization; the response time of a single thread may not improve and can worsen because it shares resources.

PLAIN ENGLISH

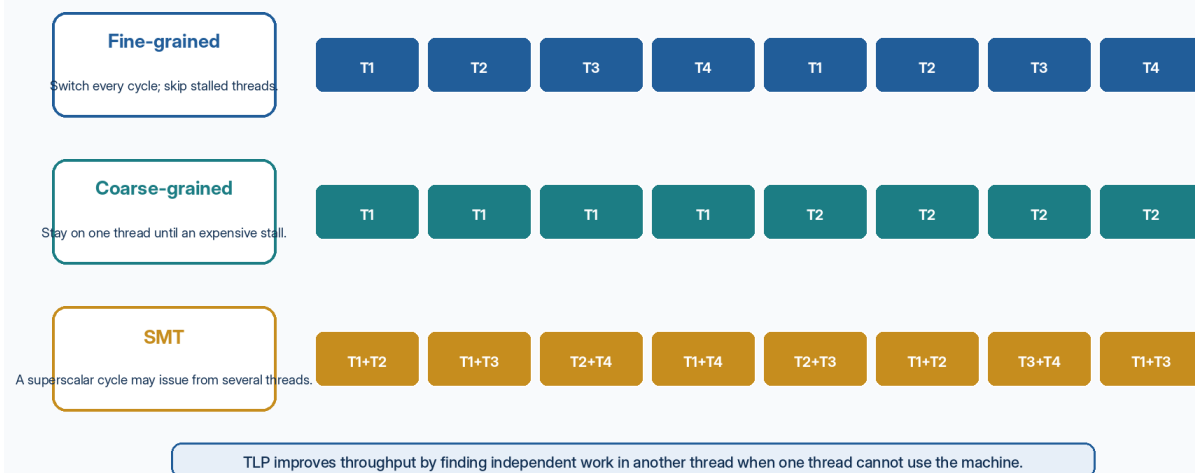
ILP finds several independent jobs inside one person's task list. TLP keeps several people's task lists and works on another list when one person is waiting.

15.2 What is replicated and what is shared

A hardware context needs a separate PC and architectural register state, plus control/status state such as exception context. Multithreaded processors share expensive structures such as pipelines, functional units, caches, translation structures, and memory interfaces to varying degrees. An out-of-order simultaneous-multithreading design also needs thread identification on ROB, rename, queue, and prediction entries so values and recovery actions remain separated.

Three multithreading strategies

Each row shows which thread supplies instructions over time.



How coarse-grained, fine-grained, and simultaneous multithreading share a processor.

15.3 Coarse-grained multithreading

Coarse-grained multithreading executes one thread for many cycles and switches when a costly event occurs, such as a long cache miss. It avoids frequent switching overhead and gives one thread full pipeline bandwidth between events. However, the pipeline may need to be emptied or refilled at the switch, and short stalls do not trigger useful overlap.

15.4 Fine-grained multithreading

Fine-grained, or interleaved, multithreading selects a different ready thread every cycle, often round-robin. Instructions from each thread are spaced apart, which can hide pipeline and memory latencies without complex forwarding

between immediately adjacent instructions of that thread. It requires several active contexts and fast thread selection. A single thread obtains only a fraction of issue slots when many threads are active, so individual-thread latency may increase.

15.5 Simultaneous multithreading

SMT combines a superscalar out-of-order processor with several hardware threads. In the same cycle, issue slots may be filled by instructions from different threads. This uses both kinds of unused capacity: vertical waste, when a whole cycle has few ready operations, and horizontal waste, when some slots in a wide cycle are empty. Threads share the instruction window and functional units, while rename maps, PCs, architectural state, and recovery bookkeeping remain logically distinct.

SMT can deliver high throughput, but threads compete for caches, queues, branch-predictor entries, ROB entries, memory bandwidth, and execution units. Fairness and quality-of-service policies may be required. A branch misprediction or exception in one thread must flush only that thread's younger work, not another thread's independent instructions.

15.6 Multicore

A multicore processor contains several processing cores on one chip. Each core has at least its own execution pipeline and architectural context and may itself support multithreading/SMT. Caches can be private at upper levels and shared at lower levels. Multicore scales thread throughput by replicating cores rather than only sharing one core's execution slots. It requires parallel software and introduces cache-coherence, synchronization, interconnection, and memory-bandwidth challenges.

Technique	Issue in a cycle	Switch trigger / scheduling	Primary benefit	Main drawback
Coarse-grained MT	One thread.	Switch on a long-latency stall.	Hides expensive misses with modest complexity.	Pipeline switch cost; short stalls remain visible.
Fine-grained MT	Usually one thread, but thread changes each cycle.	Cycle-by-cycle interleaving.	Hides many latencies and keeps pipeline occupied.	One thread receives fewer cycles.
SMT	Several threads may issue together.	Dynamic selection from all ready threads.	Fills unused superscalar slots; high utilization.	Complex sharing, contention, and fairness.
Multicore	Several cores issue independently.	OS/software maps threads to cores.	True parallel core execution and scalable throughput.	Coherence, synchronization, bandwidth, parallel-code need.

15.7 Throughput versus latency

Multithreading is chiefly a throughput technique. When one thread waits, another uses the hardware. A workload completing more total instructions or requests per second is a success even if one selected thread does not finish sooner. This distinction connects directly to Chapter 2: system throughput and individual response time must not be confused.

15.8 Exam-ready answer

EXAM-READY ANSWER

Thread-level parallelism executes several independent instruction streams. A multithreaded core replicates private thread state such as PC and architectural registers while sharing pipelines, functional units, and caches. Coarse-grained multithreading switches on a long stall; fine-grained multithreading interleaves threads cycle by cycle; simultaneous multithreading allows a wide superscalar core to issue instructions from several threads in the same cycle. These techniques improve throughput and resource utilization by hiding stalls, but add context/control state and cause competition for caches, queues, execution units, and bandwidth. Multicore instead replicates complete cores and introduces software parallelism, synchronization, and cache-coherence requirements.

Lecture grounding: 14 - thread-level parallelism.pdf, slides 2-20; 01 - computer architecture.pdf for throughput versus response time; 08 - cache.pdf for multicore coherence context.

Chapter 16 - Exam-ready model answer bank

EXAM PRIORITY: VERY HIGH

Use these as recall targets: each answer is complete enough to earn theory marks without becoming unfocused.

The following answers are deliberately written in the order the professor usually rewards: definition and purpose, architecture or classification, read/write behavior over time, advantages and limitations, then an example or consequence. Learn the structure rather than memorizing every sentence. When the question is split into numbered points, visibly number your answer in the same way.

SCORING DISCIPLINE

Mirror the numbered subquestions. Write visible labels such as 1) architecture, 2) behavior, 3) advantages. A correct paragraph that hides one requested point is harder to award than the same content clearly mapped to the prompt.

16.1 Dependencies and when they cause hazards

MODEL ANSWER

Program dependencies are true data, name, memory, and control dependencies. A true dependence exists when a later instruction reads a value written by an earlier one; it may cause a RAW hazard if the consumer reaches its read/use point before the producer value is available. An antidependence exists when an earlier instruction reads a register that a later instruction writes; it may cause WAR only when reads can occur out of order. An output dependence exists when two instructions write the same name; it may cause WAW when writes can complete out of order. RAR is not a hazard because neither instruction changes the value. Memory dependence is the analogous relation between loads/stores whose effective addresses may be equal. A control dependence exists when execution of an instruction depends on a branch result; prediction/speculation can violate the control order only if data flow and exception behavior remain correct. Register renaming removes register WAR and WAW, but not RAW.

Small code examples to include when requested

```
RAW: add x5, x1, x2 ; sub x6, x5, x3
WAR: add x5, x1, x2 ; sub x1, x3, x4
WAW: add x5, x1, x2 ; sub x5, x3, x4
RAR: add x5, x1, x2 ; sub x6, x1, x3
```

Lecture grounding: 07 - ilp.pdf; 04 - pipeline.pdf; supplied 25 February 2026 dependency and hazard prompts.

16.2 Pipeline hazards

MODEL ANSWER

A pipeline hazard is a condition preventing the next instruction from proceeding in its intended cycle. Structural hazards arise when simultaneous instructions require the same nonduplicated resource, for example one unified memory for instruction fetch and data access. Data hazards arise from operand timing: RAW is read after write and corresponds to a true dependence; WAR is write after read and corresponds to an antidependence; WAW is write after write and corresponds to an output dependence. WAR and WAW do not normally occur in a simple five-stage in-order pipeline because reads and writes occur in order, but they may occur with out-of-order execution or unequal latencies. Control hazards arise because the next PC is unknown after a branch. Solutions include duplicating/pipelining resources, stalling/interlocks, forwarding, compiler scheduling, register renaming, and branch prediction with flushing on a wrong prediction.

Lecture grounding: 04 - pipeline.pdf; 05 - multicycle.pdf; 07 - ilp.pdf; supplied 18 and 25 February 2026 hazard prompts.

16.3 Forwarding and the load-use hazard

MODEL ANSWER

Forwarding, or bypassing, reduces RAW stalls by routing a produced result directly from a later pipeline stage to the input of a dependent execution stage, without waiting for the register-file write-back. For an ALU operation, the result is normally available at the end of EX and can be forwarded to a following instruction's EX inputs. A load obtains its data only after MEM, so an immediately following consumer usually reaches EX one cycle too early and requires a one-cycle load-use stall even with forwarding. The interlock freezes the consumer and earlier stages, inserts a bubble into execution, and then the loaded value is forwarded. Forwarding does not remove a true dependence; it only shortens the communication delay.

Lecture grounding: 04 - pipeline.pdf, forwarding and load-delay sections.

16.4 Pipelined versus unpipelined functional units

MODEL ANSWER

Functional-unit latency is the number of cycles separating the start/result timing relevant to a dependent operation. Initiation interval is the minimum number of cycles between starts of two operations on that unit. A pipelined unit can overlap operations, so its initiation interval can be one even when latency is several cycles. An unpipelined unit remains occupied for the operation and usually has an initiation interval close to its latency. Consequently, two independent multiplies may start in consecutive cycles on a pipelined multiplier, while two divides must be separated when the divider is unpipelined. Multi-cycle unequal latencies create longer RAW waits, structural conflicts, and possible WAW completion hazards.

Lecture grounding: 05 - multicycle.pdf.

16.5 Static and dynamic branch prediction

MODEL ANSWER

Branch prediction reduces control-hazard stalls by selecting a likely next PC before the real branch result is available. Static prediction is fixed by the compiler or a hardware rule and does not adapt during the current execution; examples are always taken, always not taken, backward taken/forward not taken, and profile-based prediction. It requires little run-time hardware but normally has lower accuracy when behavior varies. Dynamic prediction is implemented by hardware structures such as BHTs and BTBs and updates its state from actual outcomes. It adapts to each program run and usually improves accuracy, but adds storage, access time, aliasing, power, and recovery hardware. On a misprediction, younger wrong-path instructions are flushed and fetch restarts at the correct PC.

Lecture grounding: 09 - branch prediction.pdf; supplied 28 January 2020 branch-prediction prompt.

16.6 General BHT and one-bit predictor

MODEL ANSWER

A Branch History Table is a memory of N entries containing m -bit direction-prediction states. The branch PC indexes the table using $\log_2(N)$ low-order address bits after excluding instruction-alignment bits. The selected state predicts Taken or Not Taken; after execution, the actual outcome updates the same entry. Because a simple BHT does not store a full PC tag, distinct branches may alias and influence one another. In a one-bit BHT, 0 predicts Not Taken and 1 predicts Taken, and the bit is replaced by the latest actual outcome. This adapts quickly but one exceptional result reverses the prediction, causing two boundary errors for a repeatedly taken loop across successive invocations.

Lecture grounding: 09 - branch prediction.pdf; 09e1 BHT examples; supplied 2020, 2021, and 2026 BHT questions.

16.7 Two-bit saturating and (2,2) correlating predictors

MODEL ANSWER

In a two-bit saturating predictor, each BHT entry is a counter with states 0-3. States 0 and 1 predict Not Taken; states 2 and 3 predict Taken. A taken outcome increments the counter and a not-taken outcome decrements it, with saturation at the endpoints. Two bits provide hysteresis, so one exceptional outcome does not necessarily reverse a strong prediction. A (2,2) correlating predictor additionally keeps a two-bit global history register. Each indexed branch entry contains four two-bit counters, and the history selects which counter is used. After resolution the selected counter and global history are updated. Correlation distinguishes different recent branch contexts, improving some patterns at the cost of more state, warm-up, and aliasing.

Lecture grounding: 09 - branch prediction.pdf; supplied 28 January 2020 (2-bit and (2,2)) prompt.

16.8 Branch Target Buffer

MODEL ANSWER

A Branch Target Buffer is a cache-like structure used to provide a taken-branch target during instruction fetch. It is indexed or associatively searched with the current PC. A valid matching entry contains branch-PC tag information and the predicted target address; a hit sends that target to the next-PC logic, commonly together with or under control of a direction predictor. On a miss, fetch follows the sequential PC. When the branch resolves, the entry is allocated or updated with the actual target according to the specified design. The PC tag is needed to avoid treating an unrelated instruction that maps to the same index as the stored branch. A BHT predicts direction, whereas a BTB supplies the target.

Lecture grounding: 09 - branch prediction.pdf; supplied 25 February 2026 BTB prompts.

16.9 Instruction-level parallelism and scheduling

MODEL ANSWER

Instruction-level parallelism is the degree to which instructions are independent and can overlap. It helps a pipeline approach a low CPI because independent work can execute while other instructions wait. Static scheduling is performed by the compiler, producing simpler hardware but being limited by unknown addresses and run-time delays and often tuning code to a specific latency model. Dynamic scheduling is performed by the processor, using more complex hardware but adapting to actual operands, cache behavior, and functional-unit availability while preserving binary portability. Both methods must preserve true data dependencies, possible memory dependencies, exception behavior, and the correct data flow after branches.

Lecture grounding: 07 - ilp.pdf; supplied 28 January 2020 ILP prompt.

16.10 Loop unrolling

MODEL ANSWER

Loop unrolling is a static code transformation, normally performed by the compiler, that replicates the loop body and increases the index by the unroll factor so that fewer loop iterations are executed. It reduces branch and loop-control overhead and enlarges the scheduling region, exposing independent operations from different original iterations and therefore more ILP. Its disadvantages are larger code, greater register pressure, more complex cleanup code for a nonmultiple trip count, and possible instruction-cache cost. It is legal only when dependencies between iterations are preserved; variables and registers may need renaming to avoid false dependencies.

Example transformation

```
for (i=0; i+3<N; i+=4) {
    y[i]   = x[i]   + 5;
    y[i+1] = x[i+1] + 5;
    y[i+2] = x[i+2] + 5;
    y[i+3] = x[i+3] + 5;
}
```

Lecture grounding: 07 - ilp.pdf; supplied 2020 and 2022 loop-unrolling questions.

16.11 Reservation stations

MODEL ANSWER

Reservation stations are distributed buffers placed before functional units in Tomasulo's algorithm. They hold issued instructions until their source operands and execution unit are ready, allowing younger independent instructions to proceed instead of being blocked. An entry contains Busy, operation Op, ready operand values Vj and Vk, producer tags Qj and Qk for missing operands, and an address/immediate field A; speculative designs also associate a destination ROB tag. Issue writes the entry and performs renaming. The CDB supplies values whose tags match Qj/Qk. The scheduler reads a station when all operands are ready, and the entry is released after execution/result handling. Tags remove register WAR and WAW hazards but preserve RAW flow.

Lecture grounding: 10 - dynamic scheduling.pdf; supplied 3 February 2026 reservation-station prompt.

16.12 Common Data Bus

MODEL ANSWER

The CDB is the broadcast network on which completed functional units and the load/memory unit place a producer tag and result. All reservation stations monitor it and capture the value when the tag matches Qj or Qk. The register file or rename/status table also observes it, and in a speculative machine the ROB stores the value and marks the destination entry ready. A result is written when execution is complete and the producer wins bus arbitration. With one CDB, simultaneous completions serialize because only one result can broadcast per cycle. Multiple CDBs improve write-result and dependent wake-up bandwidth, but require extra ports, comparators, arbitration, wiring, power, and area.

Lecture grounding: 10 - dynamic scheduling.pdf; 11 - hardware-based speculation.pdf; supplied 2021 and 3 February 2026 CDB prompts.

16.13 Tomasulo operation

MODEL ANSWER

Tomasulo's algorithm implements dynamic scheduling with reservation stations, register-result tags, functional units, load/store buffers, and a CDB. At issue, performed in program order, a free station is allocated, ready source values or pending producer tags are copied, and the destination register is renamed to the new producer. During execute, the instruction waits until all operands and a suitable functional unit are available, then may run out of order. During write result, the producer broadcasts its tag and value on the CDB, waking every matching consumer and conditionally updating the current destination mapping. Renaming eliminates register WAR and WAW hazards, while tags enforce RAW. Basic Tomasulo does not by itself guarantee precise exceptions; an ROB adds speculative in-order commitment.

Lecture grounding: 10 - dynamic scheduling.pdf.

16.14 Reorder Buffer

MODEL ANSWER

The ROB is a circular FIFO that lets instructions execute and complete temporarily out of order but commit architectural effects in program order. An entry is allocated at successful issue and stores at least instruction type, destination, value, and ready state, often with exception and branch information. Functional units write completed values through the CDB; dependent instructions may use those values before commitment. The commit unit examines only the oldest entry. A ready register instruction updates the architectural register, a ready store updates memory, and the entry is deallocated. Exceptions are recorded and taken only at the head, while a mispredicted branch flushes younger speculative entries. The ROB therefore supports speculation, recovery, and precise exceptions.

Lecture grounding: 11 - hardware-based speculation.pdf; supplied 2021 and 2022 ROB questions.

16.15 Exceptions and precise state

MODEL ANSWER

Exceptions may be internal or external, synchronous or asynchronous, requested or coerced, maskable or nonmaskable, and resumable or terminating. An exception is precise when every instruction older than the faulting instruction has completed its architectural effects and the faulting instruction and all younger instructions have not. This gives software a state equivalent to sequential execution and permits reliable restart/debugging. In an in-order pipeline, the processor carries exception status with each instruction, suppresses its and younger writes, drains/flushes appropriately, and invokes the handler at the instruction's ordered completion point. In an out-of-order speculative processor, results and exception status are held in the ROB; the exception is serviced only when its entry reaches the head, after older entries have committed and younger entries can be flushed.

Lecture grounding: 06 - exceptions.pdf; 11 - hardware-based speculation.pdf; supplied 18 February 2026 exception prompt.

16.16 Cache architecture and mapping

MODEL ANSWER

A cache is a small fast memory exploiting temporal and spatial locality by storing copies of main-memory blocks. Each cache line contains the block plus directory state such as tag and valid bit, and a write-back cache also has a dirty bit. The byte address is split into offset, set index, and tag. The index selects one set; the stored tags in that set are compared, and a valid match gives a hit. Direct mapping allows one line per block, W-way set associativity allows W positions in one set, and full associativity allows any line. A miss loads the block and may use LRU, FIFO, LFU, or another policy to select a victim. Write-through updates the next level on every write; write-back defers it until a dirty victim is evicted.

Lecture grounding: 08 - cache.pdf.

16.17 Multiple issue, superscalar, and VLIW

MODEL ANSWER

A width-W multiple-issue processor can ideally start W instructions per cycle, giving peak IPC W and CPI $1/W$. A dynamic superscalar processor fetches and renames several sequential instructions, uses reservation stations and several functional units to execute ready operations out of order, and commits through an ROB in order. It adapts to run-time delays but needs complex dependency checking, multiported structures, result bandwidth, and recovery. VLIW instead encodes several compiler-scheduled operations in one long word, reducing run-time issue complexity. Its costs are wide code, empty slots, register pressure, poor adaptation to unpredictable latency, demanding compilation, and binary dependence on resource/latency assumptions.

Lecture grounding: 12 - multiple-issue processors.pdf; 13 - VLIW processors.pdf.

16.18 Thread-level parallelism

MODEL ANSWER

Thread-level parallelism uses multiple instruction streams. A multithreaded core duplicates private state such as PCs and architectural registers while sharing execution resources and caches. Coarse-grained multithreading switches after a long stall; fine-grained multithreading changes thread each cycle; simultaneous multithreading may issue instructions from several threads in the same superscalar cycle. These techniques hide stalls and raise throughput/resource utilization, but can increase one thread's latency and create contention for caches, queues, units, and bandwidth. Multicore replicates complete cores and adds the need for parallel software, synchronization, interconnection, and cache coherence.

Lecture grounding: 14 - thread-level parallelism.pdf.

16.19 How to adapt a model answer to the exact wording

Question verb	What the examiner expects
Define / explain what	A precise first sentence stating purpose and identity.
Describe architecture	Named structures, fields, placement, inputs, and outputs.
Describe behavior	Chronological read/write/update/release sequence.
List advantages/disadvantages	Mechanism-specific effects, not generic faster/slower claims.
Give an example	Two or more assembly instructions or one concrete state transition.
Compare	Use the same dimensions for both alternatives.
Determine final content	Show the old state, prediction/action, actual event, and new state for every step.

Chapter 17 - Mechanical solving recipes for theory-linked exercises

EXAM PRIORITY: VERY HIGH

When recall fails, follow a fixed sequence and write every intermediate state.

Default answer skeleton

Definition -> Architecture -> Behavior over time -> Correctness role
-> Advantages/disadvantages -> Example

For a 3-part prompt, visibly write:

- 1) ...
- 2) ...
- 3) ...

17.1 Universal five-pass method

1. Pass 1 - Copy assumptions: stage order, latencies, forwarding, issue width, table size, initial states, block size, associativity, and replacement convention.
2. Pass 2 - Normalize units: byte address versus block number, cycles versus instructions, bits versus bytes, register value versus producer tag.
3. Pass 3 - Write dependencies/state before timing: producers/consumers, branch table indices, cache set/tag, or ROB/RS tags.
4. Pass 4 - Process chronologically. Never use a state that is updated later in the same step unless the stated model permits it.
5. Pass 5 - Audit invariants: no consumer before value availability, no two instructions in an unpipelined unit, no out-of-order commit, no cache replacement outside the addressed set, and no predictor update before prediction.

WHY THIS GAINS MARKS

Even when the final table is incomplete, labeled intermediate calculations reveal correct method and make partial credit possible. A final number with no derivation gives the examiner nothing to rescue.

17.2 Dependency and hazard classification recipe

1. For each instruction, write its READ set and WRITE set.
2. For every ordered pair I_{old} before I_{young} , compare the sets.
3. $WRITE(old)$ intersects $READ(young)$ -> true dependence; possible RAW.
4. $READ(old)$ intersects $WRITE(young)$ -> antidependence; possible WAR.
5. $WRITE(old)$ intersects $WRITE(young)$ -> output dependence; possible WAW.
6. $READ/READ$ alone -> RAR, not a hazard.
7. Then ask whether the machine timing can reverse the required read/write order. In a simple in-order five-stage pipeline, WAR and WAW normally cannot happen; with out-of-order execution or unequal latencies, they can.
8. Check memory addresses separately. Unknown or possibly equal addresses create a potential memory dependence.

Mark reads and writes first

I1: flw f1, 0(x1)	R={x1}	W={f1}
I2: fmul f2, f1, f3	R={f1, f3}	W={f2}
I3: fadd f1, f4, f5	R={f4, f5}	W={f1}

I1 -> I2: f1 in W1 & R2 -> true dependence / RAW
I2 -> I3: f1 in R2 & W3 -> antidependence / possible WAR
I1 -> I3: f1 in W1 & W3 -> output dependence / possible WAW

17.3 Pipeline table recipe

The supplied pipeline exercises use stage letters such as F, D, E, M, W and multi-cycle execution units. The exact timing convention is given by the exercise or reference solutions, so copy it before filling the table. The safe strategy is to schedule one instruction at a time but repeatedly check resource and dependency constraints against all older instructions.

1. Write the instruction list and annotate every source, destination, operation unit, and latency.
2. Draw arrows from each producer to each consumer. Mark loop-carried dependencies separately.
3. Place fetch/decode in program order. If decode/issue is blocked, later fetch/decode normally cannot pass it in the course model.
4. Determine the earliest cycle each source is usable. With forwarding, use the producer-result availability stated by the pipeline; without it, wait for register write/read timing.
5. Place execution no earlier than operand readiness and functional-unit availability.
6. For a pipelined unit, enforce its initiation interval, not its full latency, between independent starts. For an unpipelined unit, reserve it for all execution cycles.
7. Check write-result/write-back bandwidth. Multiple operations finishing together may serialize if only one write port/CDB exists.
8. Apply branch behavior. If the delay slot is disabled, do not intentionally execute the sequential slot as architecturally useful work; account for the stated stall/flush until the correct next PC is known.
9. For a loop, finish the first iteration carefully, then identify which next-iteration instructions can overlap and which loop-carried dependencies restrict them.
10. Total cycles equal the last occupied cycle, not the sum of the per-instruction latencies.

VALUE-AVAILABILITY QUESTIONS TO ASK

- Where is the value physically produced - end of ALU EX, end of memory stage, or end of a multi-cycle FP unit?
- Where does the consumer need it - beginning of EX, store-data input, branch comparison, or decode/register read?
- Is there a forwarding path between those points?
- Does the table convention allow a result produced at the end of cycle c to be used at the beginning of cycle $c+1$?

LOAD-USE SHORTCUT

In a classic five-stage pipeline with forwarding, an ALU result can usually feed the next instruction, but a load followed immediately by its consumer needs one bubble because the memory value appears too late. For the professor's multi-cycle tables, always use the explicit result-availability convention shown in the current exam/reference solution.

17.4 Branch delay slot recipe

Statement in question	How to treat the instruction after the branch
Delay slot enabled	The architectural instruction in the slot executes regardless of whether the branch is taken; it should be useful if safely schedulable, otherwise a NOP.
Delay slot disabled	There is no architecturally guaranteed useful slot. Sequential instructions fetched before resolution are speculative/wrong-path and are discarded if necessary.
Predict not taken	Continue sequential fetch; flush if branch turns out taken.
Predict taken	Fetch target when target is available; flush if branch turns out not taken.
No prediction / freeze	Stop or insert bubbles until the branch direction/target is known.

DO NOT CONFUSE

A branch delay of one cycle is a timing property. An enabled architectural delay slot means the following instruction is defined to execute. The professor can state a one-cycle branch delay while saying the delay slot is not enabled; then the timing penalty exists but the sequential instruction is not guaranteed as useful program work.

17.5 One-bit BHT table recipe

1. Compute alignment shift $a = \log_2(\text{instruction bytes})$. For four-byte instructions, $a=2$.
2. Compute index = (branch PC $\gg$ a) mod N.
3. Read old bit: 0 -> predict NT, 1 -> predict T.
4. Compare to actual T/NT and record correct/wrong.
5. Write actual outcome into that entry.
6. Continue in execution order; collisions use the state left by earlier branches.

PC	Index	Old	Prediction	Actual	Correct?	New
0x...						
0x...						
0x...						

17.6 Two-bit BHT recipe

1. Use the same PC-index calculation.
2. Read old counter. 0/1 -> NT; 2/3 -> T.
3. Mark correctness against the actual result.
4. If actual T, new = min(3, old+1). If actual NT, new = max(0, old-1).
5. Never wrap 3 to 0 or 0 to 3; it is saturating.

Four transitions worth memorizing

```

old=1, actual=T -> prediction NT (wrong), new=2
old=2, actual=NT -> prediction T (wrong), new=1
old=3, actual=T -> prediction T (correct), new=3
old=0, actual=NT -> prediction NT (correct), new=0

```

17.7 Correlating predictor recipe

1. Write the initial global-history register and state which end receives the new outcome.
2. Use the branch PC to select the branch entry or PC component of the index.
3. Use the old global history to select the counter. Do not update history first.
4. Predict from the selected old counter and compare with the actual result.
5. Update only the selected counter with saturation.
6. Shift the actual outcome into global history exactly according to the diagram.
7. Repeat. Two executions of the same PC can use different counters because history changed.

17.8 BTB table recipe

BTB exercises differ in whether an entry stores the full branch PC, a tag, a target only, and whether a matching entry alone implies Taken. Copy the entry format and update policy from the question. The common supplied format uses an initially zero table and stores branch and target information for taken branches.

1. Compute the BTB index from the branch PC using the specified number of entries and instruction alignment.
2. Read the entry before updating. Check valid/tag/address match, not only whether the row is nonzero.
3. On a matching hit, record the predicted target and the implied/associated direction.
4. On a miss, record sequential/not-taken behavior unless the question specifies another predictor.
5. Compare with actual direction and target.

6. Allocate, replace, invalidate, or update only according to the stated scheme. If two branches alias, the later update may replace the earlier entry.
7. At the end, reproduce every row, including untouched zero rows.

Index	Valid/tag/branch PC	Stored target	Lookup result / update
0			
1			
...			

17.9 Cache configuration recipe

1. Write A, C, B, and W with units.
2. Lines $N=C/B$. Sets $S=N/W$.
3. Offset bits $b=\log_2(B)$. Index bits $s=\log_2(S)$. Tag bits $t=A-s-b$.
4. If given a byte address: $\text{block}=\text{address} // B$, $\text{offset}=\text{address} \bmod B$.
5. $\text{Set}=\text{block} \bmod S$. $\text{Tag}=\text{block} // S$.
6. Search only the selected set. A valid equal block/tag is a hit.
7. On a miss, use an invalid line first; otherwise choose a victim by the exact replacement state.
8. If the victim is dirty in a write-back cache, include its write-back before refill.
9. Update replacement/frequency metadata after both hits and insertions according to the exercise convention.
10. Count hits/misses and copy the complete final cache, not only changed rows.

The unit fork

Given a BLOCK number q and S sets:

$\text{set} = q \bmod S$

$\text{tag} = q // S$

Given a BYTE address a and block size B :

$q = a // B$

$\text{offset} = a \bmod B$

$\text{set} = q \bmod S$

$\text{tag} = q // S$

17.10 LRU state-update recipe

Use the problem's exact numbering. A frequent supplied convention labels the most recently used way 0 and the least recently used way $W-1$. To access a line whose old position is k , lines more recent than it (labels below k) age by one, the accessed line becomes 0, and lines older than it remain in relative order. On replacing the LRU line, the surviving lines age by one and the inserted line becomes 0. This permutation keeps each label unique from 0 to $W-1$.

LRU convention where 0 = MRU

Four-way set, labels [0,1,2,3]

Hit line with label 2:

label 0 $\rightarrow$ 1

label 1 $\rightarrow$ 2

accessed label 2 $\rightarrow$ 0

old label 3 $\rightarrow$ 3

Miss:

victim old label 3

old 0/1/2 $\rightarrow$ 1/2/3

inserted line $\rightarrow$ 0

17.11 LFU state-update recipe

1. On a hit, increment the requested line's frequency counter.
2. On a miss with an invalid way, use it and initialize its counter as specified, usually zero.
3. On a full-set miss, find the minimum frequency.
4. If several ways share that minimum, replace the oldest one indicated by the pointer/arrow or stated age convention.

5. After replacement, initialize the new counter and update the tie-breaking oldest pointer exactly as described.
6. Do not reset unrelated frequency counters.

17.12 Amdahl-law recipe

1. Normalize the original execution time to 1 unless actual time is needed.
2. Identify f as the fraction of ORIGINAL time affected, not the fraction after improvement.
3. Replace that portion by f/S .
4. New normalized time = $(1-f)+f/S$.
5. Overall speedup = $1/\text{new time}$.
6. Check the upper bound: as S approaches infinity, maximum speedup = $1/(1-f)$.

Worked Amdahl example

40% of time is accelerated by 5x:
 $f = 0.40$, $S = 5$
 $T_{\text{new}}/T_{\text{old}} = 0.60 + 0.40/5 = 0.68$
 Speedup = $1/0.68 = 1.4706x$
 Maximum possible even with infinite enhancement = $1/0.60 = 1.6667x$

17.13 CPU-time comparison recipe

FORMULA

CPU time = instruction count $\times$ average CPI $\times$ clock-cycle time = instruction count $\times$ average CPI / clock rate.
 Performance ratio A/B = $\text{time}_B/\text{time}_A$.

1. Convert every clock rate to period or use the division form consistently.
2. If several instruction classes exist, compute total cycles as $\sum(\text{IC}_i \times \text{CPI}_i)$.
3. Include stalls/misses when CPI components are provided.
4. Compare execution times, not clock rates alone.
5. State which machine is faster and by what factor, with the ratio in the correct direction.

17.14 Tomasulo snapshot recipe

1. Process issue in program order. Check for a free RS/load-store buffer and, with speculation, a free ROB entry.
2. Allocate destination tag first conceptually, then read each source: ready value $\rightarrow V$; pending producer $\rightarrow Q$ tag.
3. Update the RAT/register-status destination to the new youngest producer.
4. For every CDB broadcast, scan all Q_j/Q_k fields for matching tags, copy value into V , and clear the Q .
5. Start only stations with all operands ready and an available compatible unit; apply functional-unit latency/initiation interval.
6. When execution finishes, arbitrate for the CDB. Arithmetic completion and broadcast may be different cycles.
7. With an ROB, place the value/ready flag in the ROB; do not update architectural state yet.
8. Commit only ready head entries. Stores write memory here; exceptions/mispredictions recover here according to the model.
9. After each cycle, verify that every pending Q tag refers to a real in-flight producer.

RS	Busy	Op	Vj	Vk	Qj	Qk	A / dest
Add1							
Mul1							
Load1							

ROB	Instruction	Destination	Value/status	Ready
1				
2				
3				

17.15 Theory-answer construction recipe

1. Sentence 1: definition and purpose - "X is a structure/technique used to..."
2. Architecture: name location, entries/fields, inputs, outputs, and connected modules.
3. Timeline: state exactly when it is allocated/read/written/updated/released.
4. Correctness: say which hazards/dependencies/state-order property it handles.
5. Trade-off: one concrete benefit and one concrete hardware/compiler cost.
6. Example: a two-instruction dependence, a predictor transition, or a one-line numerical calculation.
7. Audit against every numbered request before moving on.

Lecture grounding: Recipes synthesized from lectures 01, 04-11 and the supplied solved/unsolved examination formats. Exact numerical timing and replacement conventions must always follow the current paper.

Chapter 18 - Last-day recall sheets, likely variations, and self-test

EXAM PRIORITY: VERY HIGH

Use this chapter for closed-book retrieval, not passive rereading.

18.1 Thirty-six one-line recall targets

Term	Recall sentence
Architecture	ISA + organization + hardware implementation.
CISC	Complex/variable instructions and rich addressing for compact code; more complex control.
RISC	Simple regular load-store instructions, often fixed length and pipeline-friendly.
Performance	Performance is inverse execution time; throughput and response time are distinct.
CPU time	IC x average CPI x clock-cycle time.
Amdahl	Speedup = $1 / ((1-f) + f/S)$; the unenhanced fraction sets the limit.
Dynamic power	Approximately proportional to $C \times V^2 \times f$.
Pipeline	Overlaps stages of several instructions to raise throughput, not reduce one instruction's latency.
Structural hazard	Two simultaneous operations need the same insufficient resource.
RAW	Consumer reads before older producer value is available; true dependence.
WAR	Younger write occurs before older read; antidependence.
WAW	Younger write completes before older write; output dependence.
RAR	Two reads; no hazard.
Forwarding	Directly routes a produced value to a consumer without waiting for register write-back.
Load-use	Usually still needs one stall in a classic five-stage pipeline because load data appears after MEM.
Latency	Delay until a result can be used by a dependent operation.
Initiation interval	Minimum separation between starts on one functional unit.
Precise exception	Older effects committed; faulting and younger effects absent.
ILP	Independent scalar instructions that can execute or overlap.
Loop unrolling	Replicates loop body to reduce control overhead and expose cross-iteration ILP.
Register renaming	Gives values unique names, removing register WAR and WAW but not RAW.
Cache	Small fast memory exploiting temporal and spatial locality.
Direct mapped	One possible line per memory block.
Set associative	One indexed set, then any of W ways.
Write-through	Update cache and next level on each write.
Write-back	Update cache, mark dirty, and write next level on eviction.
BHT	PC-indexed direction-history table; can alias.
Two-bit counter	0/1 predict NT; 2/3 predict T; saturating decrement/increment.
BTB	PC/tag plus target cache used early to redirect fetch.
Reservation station	Waits near a functional unit with operand values or producer tags.
CDB	Broadcasts one producer tag and value to all listeners.
Tomasulo	Issue/rename in order, execute ready operations out of order, broadcast results.

Term	Recall sentence
ROB	FIFO holding speculative results so architectural commit remains in order.
Superscalar	Dynamically issues several instructions per cycle.
VLIW	Compiler packs explicitly parallel operations into one long instruction.
SMT	A wide core issues ready instructions from several threads in one cycle.

18.2 High-yield comparison matrix

Pair	First concept	Second concept	Decisive distinction
Dependency vs hazard	Program ordering relation.	Pipeline timing conflict.	A dependency may exist without producing a hazard in a given machine.
RAW vs WAR/WAW	True value flow.	Name conflicts.	Renaming removes WAR/WAW, never RAW.
Latency vs initiation interval	Consumer-ready delay.	Same-unit start spacing.	A pipelined FU can have long latency but $II=1$.
BHT vs BTB	Predicts direction.	Provides target.	Taken/not-taken is not the same information as where to jump.
RS vs ROB	Schedules/waits near FU.	Preserves order/recovery.	RS controls execution readiness; ROB controls commit.
Execute vs commit	Produce a temporary result.	Make the result architecturally visible.	Speculative work may execute but never commit.
Static vs dynamic scheduling	Compiler before run.	Hardware at run time.	Dynamic adapts to actual delays at hardware cost.
Superscalar vs VLIW	Hardware forms groups.	Compiler encodes groups.	Who discovers/enforces parallel issue.
Fine-grained vs SMT	One thread normally chosen per cycle.	Several threads can issue in same cycle.	SMT shares width within a cycle.
Write-through vs write-back	Lower level updated immediately.	Lower level updated on dirty eviction.	Traffic versus dirty-state/eviction complexity.

18.3 Ten common traps that lose easy marks

- Calling every dependency a hazard. First name the dependency; then state the timing condition that creates its hazard.
- Writing that WAR and WAW occur in every pipeline. Explain that simple in-order timing normally prevents them.
- Saying forwarding removes dependencies. It only delivers a needed RAW value earlier.
- Using full execution latency as the start spacing of a pipelined functional unit.
- Indexing a BHT with the target address or including byte-alignment bits.
- Updating a predictor before writing down the prediction from the old state.
- Describing a BTB as only a taken/not-taken bit table.
- Treating reservation stations and the ROB as interchangeable queues.
- Allowing an out-of-order result or store to commit before an older instruction.
- Doing cache modulo arithmetic on a byte address when the question supplied or requires a block number first.

18.4 High-probability prompt variations

The supplied papers repeatedly vary a small number of components while keeping the expected answer structure stable. These are not guaranteed future questions; they are the most defensible variations to rehearse from the lecture and paper pattern.

Base topic	Likely variation to rehearse	What changes in the answer
CDB	Single versus multiple CDBs; list readers/writers; simultaneous completions.	Add arbitration/bottleneck and hardware-cost discussion.
Reservation stations	Fields, access timing, or comparison with ROB.	Separate operand waiting from ordered retirement.

Base topic	Likely variation to rehearse	What changes in the answer
ROB	Lifecycle, precise exceptions, branch flush, or store behavior.	Explicitly walk allocation -> write result -> head commit/recovery.
Hazards	Ask dependencies first or hazards first; RISC-V examples required.	Give READ/WRITE examples and state machine conditions for WAR/WAW.
BHT	8 versus 16 entries; 1-bit versus 2-bit; collisions.	Change index bits/state transition, not the basic process.
BTB	Entry fields, access stage, target collisions, final table.	Track tag/address and target rather than only direction state.
Branch prediction	Static/dynamic, saturating, correlating, gshare/gselect.	Compare storage/context and update mechanism.
Exceptions	Classification plus precise handling in pipeline and superscalar.	Give precise-state definition before mechanisms.
ILP	Static/dynamic scheduling or loop-unrolling example.	Tie every technique to independence, overhead, and code/HW cost.
Cache	Change associativity, block size, LRU/LFU, or initial contents.	Recompute sets/fields and obey exact metadata convention.
Multiple issue	Static packet versus dynamic superscalar or CDB limit.	State peak CPI/IPC, then practical bottlenecks.
TLP	Fine/coarse/SMT or SMT versus multicore.	Focus on issue granularity, replicated state, sharing, throughput.

18.5 Emergency pass-priority ladder

TIER 1 - MUST REPRODUCE WITHOUT NOTES

- Dependencies and RAW/WAR/WAW/RAR with code examples.
- Hazard categories, forwarding, load-use, and branch/control handling.
- BHT architecture/index/update, one-bit and two-bit transitions.
- BTB architecture and BHT-versus-BTB distinction.
- Reservation-station fields and timing.
- CDB readers, writers, timing, and multiple-CDB trade-off.
- ROB fields, lifecycle, in-order commit, branch flush, and precise exceptions.

TIER 2 - MUST RECOGNIZE AND RECONSTRUCT

- Pipeline/multicycle scheduling method and latency versus initiation interval.
- ILP, static versus dynamic scheduling, loop unrolling, register renaming.
- Cache field formulas, mapping, LRU/LFU, and write policies.
- Tomasulo issue/execute/write result and memory-order caveat.

TIER 3 - SECURE SUPPORTING MARKS

- CISC versus RISC, ISA principles, and RISC-V organization.
- Performance, Amdahl, power, dependability.
- Superscalar versus VLIW and TLP categories.

18.6 Closed-book self-test: questions

1. Why is a dependency not automatically a hazard?
2. Give one RISC-V RAW, WAR, and WAW example and name the corresponding dependency.
3. Why can a classic in-order five-stage pipeline have RAW but not normally WAR/WAW?
4. Where is an ALU result available for forwarding, and why is a load different?
5. Define functional-unit latency and initiation interval.
6. What is the exact definition of a precise exception?
7. How does an ROB guarantee the precise boundary?
8. What does register renaming remove, and what can it never remove?
9. Explain the complete BHT access/update sequence.

- 10. Which PC bits index a 16-entry BHT with four-byte instructions?
- 11. Draw the four two-bit counter states and transitions.
- 12. What extra context does a (2,2) predictor use?
- 13. What fields belong in a BTB entry, and why is a tag useful?
- 14. Distinguish a BHT hit/state from a BTB target hit.
- 15. List every reservation-station field and explain V versus Q.
- 16. Who writes the CDB and who reads it?
- 17. Why is one CDB a bottleneck?
- 18. Walk one instruction through Tomasulo issue, execute, and write result.
- 19. When is a ROB entry allocated, written, read, committed, and released?
- 20. Why must a speculative store wait until commit?
- 21. Calculate tag/index/offset bits from A, C, B, W.
- 22. Distinguish placement, replacement, and write policy.
- 23. What is the difference between compulsory, capacity, and conflict misses?
- 24. Explain the benefits and costs of loop unrolling.
- 25. Why can dynamic scheduling outperform static scheduling?
- 26. What limits a width-four superscalar machine from achieving IPC=4?
- 27. Compare dynamic superscalar and VLIW scheduling.
- 28. Compare coarse-grained, fine-grained, and SMT execution.
- 29. Use Amdahl's law for an enhancement affecting 30% of time with speedup 4.
- 30. Explain why deeper pipelines make branches more expensive.

18.7 Closed-book self-test: compact key

Q	Expected core
1	Dependency is a program relation; hazard exists only if machine timing would violate it.
2	Use WRITE->READ = RAW/true; READ->WRITE = WAR/anti; WRITE->WRITE = WAW/output.
3	Reads and writes are ordered at fixed stages; only a younger read can catch an older unfinished write.
4	ALU result after EX; load data after MEM, too late for immediate next EX without a bubble.
5	Latency = dependent-use delay; II = same-unit start spacing.
6	All older effects present; faulting and all younger effects absent.
7	Temporary results/exceptions wait in FIFO order and are acted on only at the head.
8	Removes register WAR/WAW; cannot remove RAW true value flow.
9	Index with branch PC excluding alignment, read old state/predict, resolve, compare, update same entry.
10	Bits [5:2].
11	00 SNT, 01 WNT, 10 WT, 11 ST; T increments and NT decrements with saturation.
12	Two recent global branch outcomes select one of four two-bit counters.
13	Valid, branch PC tag/address, target, optional direction/type; tag prevents false alias hit.
14	BHT state predicts direction; BTB match supplies an address.
15	Busy, Op, Vj/Vk, Qj/Qk, A, and destination tag; V is value, Q is pending producer.
16	Completed FUs/load unit write; RSs, register/rename state, and ROB read.
17	Only one completion can broadcast, so others wait and consumers wake later.
18	Allocate/rename -> wait/readiness and execute -> arbitrate/broadcast/wake.
19	Allocate at issue; write via CDB at completion; read for operands/commit; retire at ready head; then free.
20	Memory update is difficult to undo and externally visible; buffer address/data until oldest.

Q	Expected core
21	$S=C/(B \cdot W)$, $b=\log_2 B$, $s=\log_2 S$, $t=A-b \cdot s$.
22	Allowed location; victim choice; propagation of stores.
23	First reference; working set too large; mapping collision.
24	Fewer branches and more ILP; larger code/register pressure/remainder handling.
25	It sees actual readiness, addresses, cache misses, and unit availability.
26	Dependencies, branches, cache/front-end, unit mix, ports/CDB, window, commit, power.
27	Hardware dynamically groups normal instructions versus compiler-encoded parallel packets.
28	Switch on long stall; alternate each cycle; issue several threads in one wide cycle.
29	New time = $.70 + .30/4 = .775$; speedup = $1/.775 = 1.2903x$.
30	More younger stages/instructions lie behind an unresolved branch, increasing flush/restart cycles.

18.8 Twelve-day use plan from 4 to 15 September

Date	Primary target	Closed-book output
Sep 4	Hazards + dependency map	Write RAW/WAR/WAW/RAR and forwarding answer from memory.
Sep 5	Pipeline + multicycle	Complete one timing table and explain latency versus IL.
Sep 6	BHT one-bit/two-bit	Solve two index/update tables without notes.
Sep 7	BTB + static/dynamic prediction	Write both model answers and one final-content table.
Sep 8	Tomasulo + RS	Reconstruct fields and one multi-instruction state sequence.
Sep 9	CDB + ROB	Write readers/writers/lifecycle and precise-exception link.
Sep 10	Cache	Do address fields plus one LRU/LFU sequence.
Sep 11	ILP + unrolling + performance	Write one transformation and solve one Amdahl question.
Sep 12	Superscalar/VLIW/TLP + foundations	Complete comparison matrices from memory.
Sep 13	Mixed past theory	Answer three timed questions without looking.
Sep 14	Mixed exam simulation	Full Part I under exam timing; repair only recurring failures.
Sep 15	Recall only	Use Sections 18.1-18.7; no new topic and no passive all-night reread.

FINAL RULE

Recognition is not exam readiness. A topic counts as learned only when you can close the guide, produce its architecture and timing in your own words, and solve one variation without being shown the first step.

Lecture grounding: Priority and variation map inferred from the supplied 2020-2026 theory papers and grounded in lectures 00-14. It is a study prioritization, not a guarantee of the September paper.

Appendix A - Full lecture source map

Every explanatory chapter was derived from the Architecture lecture folder below. Slide ranges are included so a claim can be traced back to the course material. Past papers were used to choose answer depth, recurring wording, and exercise conventions; they were not used to replace the lecture theory. Where broad textbook conventions differ, the lecture wording is used first, and each numerical exercise must follow the assumptions printed on its paper.

Lecture file	Slides	Guide coverage
00 - CISC-RISC difference.pdf	1-28	ISA definition; register-memory/load-store; CISC/RISC traits; addressing examples.
01 - computer architecture.pdf	1-74	Architecture levels; markets; performance; benchmarks; Amdahl; power; dependability.
02 - introduction to isa.pdf	1-60	ISA storage models; data/addressing; control flow; registers; encoding; compiler interaction.
03 - RISC-V.pdf	1-46	RISC-V organization, registers, formats, arithmetic, loads/stores, branches, calls, pseudoinstructions.
04 - pipeline.pdf	1-87	Pipeline stages, throughput, structural/data/control hazards, stalls, forwarding, branch handling.
05 - multicycle.pdf	1-42	Multi-cycle FP units, latency, initiation interval, structural/RAW/WAW behavior, deeper pipelines.
06 - exceptions.pdf	1-26	Exception/interrupt classifications, restart, precise exceptions, pipeline and OoO handling.
07 - ilp.pdf	1-48	ILP, scheduling, dependencies, hazards, memory aliasing, loop unrolling, renaming, speculation conditions.
08 - cache.pdf	1-67	Locality, hierarchy, cache organization, mapping, replacement, write policies, multilevel/coherence.
09 - branch prediction.pdf	1-40	Static/dynamic prediction, BHT, one/two-bit counters, correlation, BTB, gselect/gshare.
10 - dynamic scheduling.pdf	1-50	Scoreboarding/Tomasulo, reservation stations, register status, CDB, loads/stores.
11 - hardware-based speculation.pdf	1-24	Speculative Tomasulo, ROB, RAT, in-order commit, stores, branch recovery, exceptions.
12 - multiple-issue processors.pdf	1-20	Static/dynamic multiple issue, superscalar front-end and bandwidth requirements.
13 - VLIW processors.pdf	1-26	VLIW packets, compiler scheduling, strengths, limits, compatibility, EPIC.
14 - thread-level parallelism.pdf	1-20	Fine/coarse-grained MT, SMT, shared/private state, throughput, multicore context.

Past-paper prompts used to calibrate emphasis

Paper / supplied material	Theory emphasis used
28 January 2020 variants	BHT variants, branch prediction, ILP, scheduling, and loop unrolling.
2021 papers	ROB architecture/lifecycle and CDB location, users, timing, and bandwidth.
2022 papers	Loop-unrolling and ROB wording.
2023-2025 exams/exercises	Pipeline timing, BHT tables, cache LRU/LFU, and Tomasulo practice.
3 February 2026	CDB, reservation stations, and RISC-V multi-cycle scheduling.
18 and 25 February 2026	Precise exceptions, dependencies/hazards, BHT and BTB final-content tables.

TRACEABILITY

Each chapter ends with a Lecture grounding line. Use it to return to the original slides when you need the professor's diagram, notation, or exact phrasing.